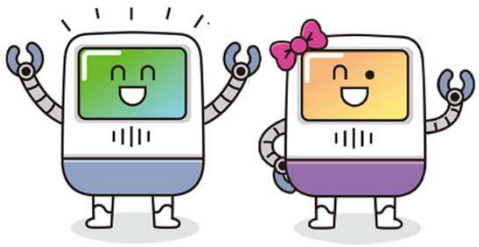
딥러닝

4장 선형 회귀와 경사하강법



04

선형 회귀와 경사하강법



- 데이터의 특성과 선형 회귀를 알아보자
  - 특성과 변수
  - 통계학이 하는 일
  - 회귀 분석이란 데이터의 분포를 설명하는 좋은 함수를 찾는 것
- 오차와 오차의 측정
  - 데이터의 분포와 경향성
  - 데이터와 정답의 차이: 오차
  - 평균 절대값 오차와 평균 제곱오차
  - 선형 회귀 직선 구하기 예시
  - 오차 곡선의 최적값을 구하는 기울기와 미분
- 경사하강법을 알아보는 실습
  - 실습하기
  - 손으로 계산하기
- 사이킷런을 이용한 선형 회귀
  - 여러 가지 머신러닝 라이브러리들
  - 간단한 데이터로 선형 회귀를 수행해 보자
  - 데이터를 시각화하고 차원을 증가시키자
  - 오차 곡면을 알아보자
  - 경사하강법과 학습률
  - 정규화와 표준화

학습목차

01 데이터의 특성과 선형 회귀를 알아보자
02 오차와 오차의 측정
03 경사하강법을 알아보는 실습
04 사이킷런을 이용한 선형 회귀

학습목표

- 데이터의 성질, 특성, 변수에 대하여 이해한다.
- 회귀 분석을 이해하고 입력된 데이터에 대한 선형적인 모델링 방법을 알아본다.
- 추정값과 실제 값의 차이인 오차와 다양한 오차 측정 기법에 대하여 이해한다.
- 경사하강법을 통하여 최적의 선형 회귀 직선을 얻는 방법을 알아본다.
- 사이킷런 라이브러리를 사용하여 선형 회귀를 수행하는 방법을 알아본다.

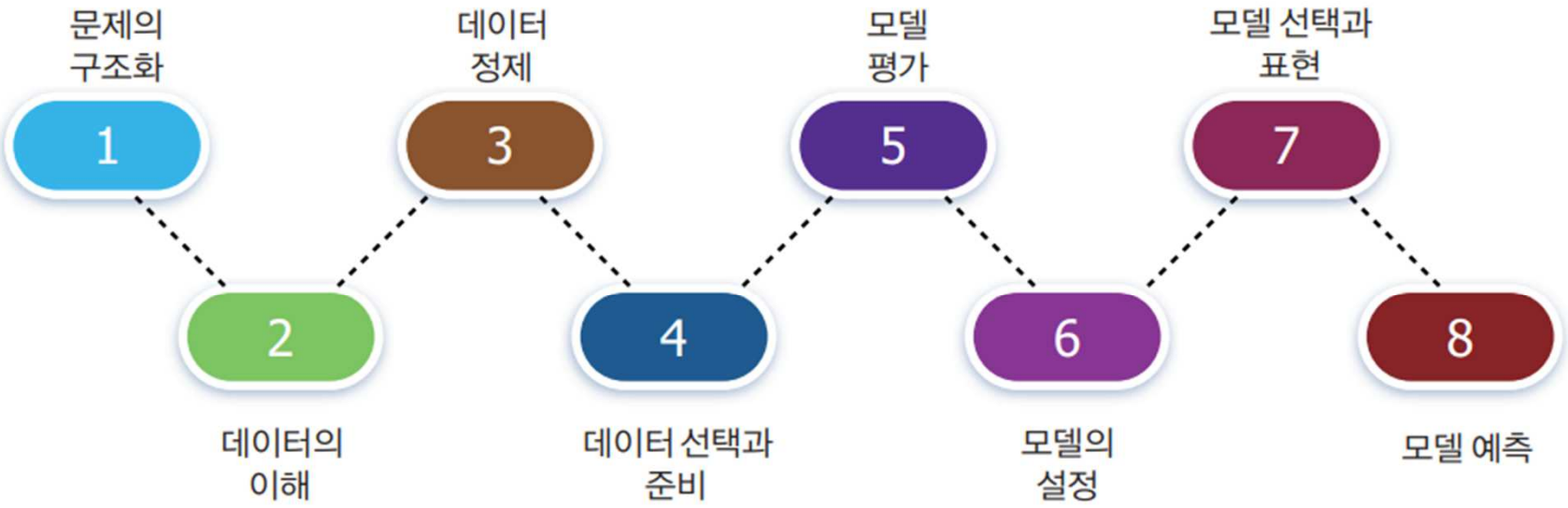
01 데이터의 특성과 선형 회귀를 알아보자

- 특성과 변수를 알아보자
  - **특성**이란, 관찰되는 현상에서 측정할 수 있는 개별적인 속성을 의미함
  - 머신러닝과 딥러닝에서 말하는 특성이라는 것은 학습의 결과를 결정하는 데에 영향을 미치는 입력 데이터를 말하기도 한다

- **사람의 키와 몸무게:** 일반적으로 키가 큰 사람이 몸무게가 더 많이 나가는 경우가 많다. 키와 몸무게는 사람의 특징을 표현하기 위한 좋은 특성이 될 수 있다.
- **개의 몸통 길이와 높이:** 말티즈처럼 작은 개와 사모예드처럼 큰 개를 구분하기 위한 방법으로 개의 몸통 길이와 높이를 입력 데이터로 주고 학습을 시킨다면 말티즈와 사모예드를 잘 구별할 수 있게 될 것이다.
- **주택 가격과 주택 면적:** 주택의 가격에 영향을 주는 특성으로는 주택의 면적, 지하철역과의 거리, 마트까지의 거리, 주택의 건축 연도, 화장실의 수와 같은 것들이 있을 것이다.
- **붓꽃의 꽃잎, 꽃받침 너비와 길이:** 꽃을 분류하는 방법 중의 하나로 꽃잎, 꽃받침의 크기에 따른 분류가 있을 수 있다.

통계학이 하는 일

- 딥러닝과 머신러닝을 이해하기 위해서는 몇 가지 통계 개념에 대한 이해가 필요하다
  - 통계학에서 문제를 해결하기 위한 구체적인 절차는 그림과 같이 문제를 구조화하고 구조화된 데이터에 대한 이해를 바탕으로 데이터를 정제하고 준비하며, 모델을 평가하고, 예측하는 절차가 있는데 이러한 절차는 머신러닝에서도 꼭 필요한 내용이기도 하다.



통계학과 머신러닝의 관련성

통계학이 하는 일

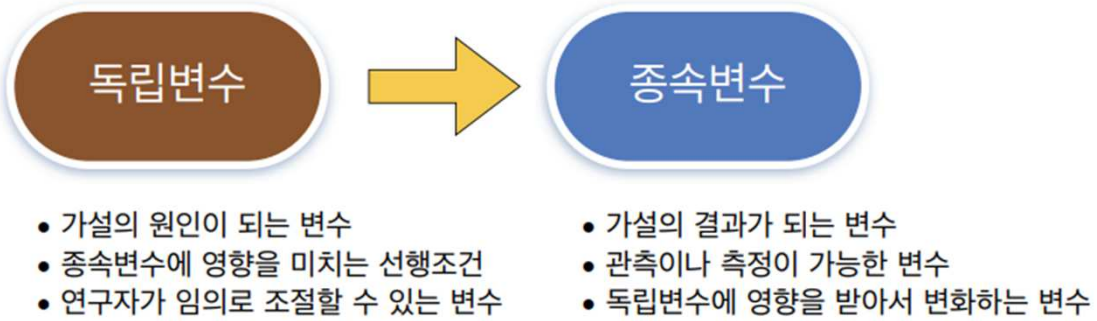
- 우리가 알아볼 첫 번째 머신러닝 방법은 **회귀 분석**(回歸分析)이다. 딥러닝을 이해하기 위하여 회귀분석에 사용되는 최적화 기법을 살펴보자
  - 회귀 분석은 딥러닝을 이해하기 위한 중요한 기초가 된다. 일상생활에서는 접하기 힘든 다소 생소한 용어인 **회귀** regression 란 어딘가로 돌아간다는 의미이다.
  - 통계 분야에서 회귀라는 용어를 처음으로 사용한 사람은 **프랜시스 골턴** Francis Galton 으로 알려져 있다.
  - 회귀 분석은 대표적인 지도학습 알고리즘으로, **관측된 데이터를 통해 독립변수와 종속변수 사이의 숨어 있는 관계를 추정하는 것**이다

용어	해설
변수	측정 결과와 조사 대상에 따라 다른 값으로 나타날 수 있는 특성 혹은 속성으로 변경될 수 있는 양이나 조건을 말한다.
독립변수	연구자가 임의로 조절할 수 있는 변수로, 실험 영역에 있어 다른 변수에 영향을 받지 않는 변수를 말한다.
종속변수	관측이나 측정이 가능한 변수로, 독립변수에 영향을 받아서 변화하는 변수를 말한다.

통계학이 하는 일

- 연구자가 특정 지역의 주택 면적과 최근 2년간 거래 가격 사이의 관계를 알아보는 경우를 생각해 보자. 이 연구를 위해서 측정해야 하는 대상에 해당하는 주택 면적과 거래 가격은 변수라고 지칭한다.
  - 주택의 면적이 큰 경우 판매 가격도 높은 경우가 많은데, 여기서 다른 변수에 영향을 덜 받는 변수인 주택의 면적은 **독립변수**가 되며, 이에 영향을 받아서 변화할 수 있는 거래 가격이 **종속변수**가 될 것이다





통계학이 하는 일

도전 문제 4.1: 독립변수와 종속변수의 구분

상 중 하

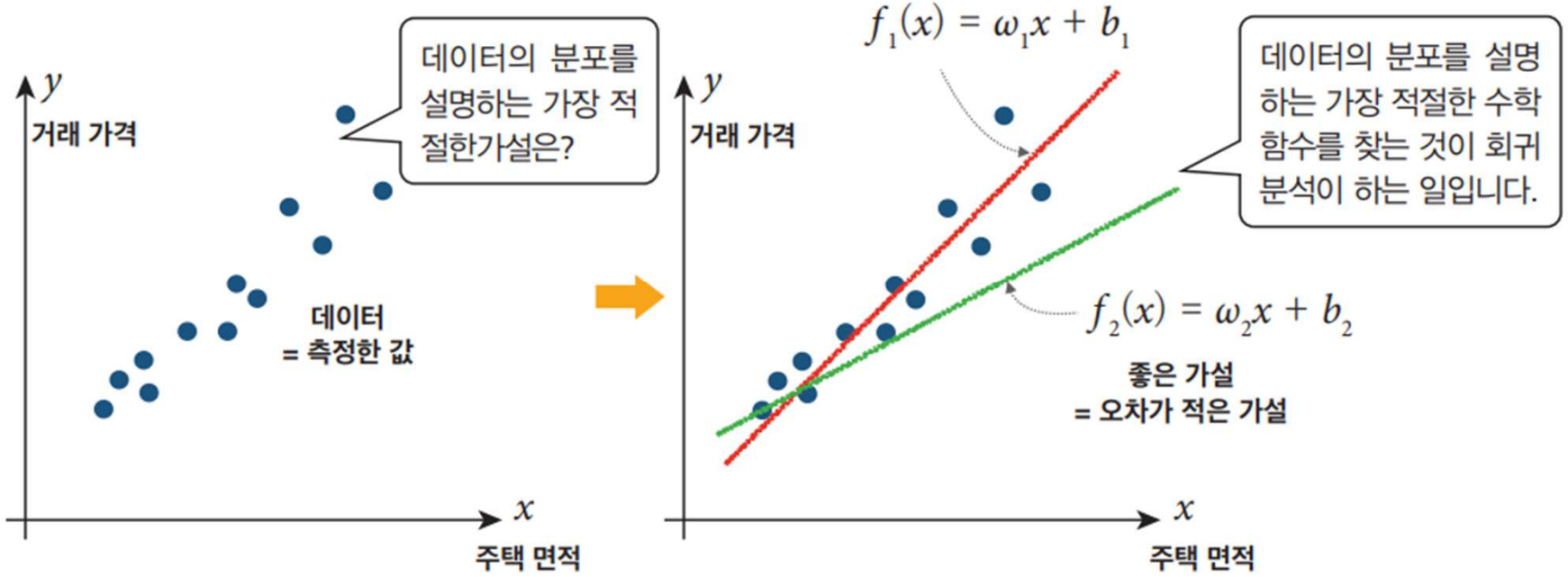


1. 다음과 같은 가설과 이에 따른 변수가 있을 경우, 이 변수를 독립변수와 종속변수로 구분해 보자.

   - 가설 1: 평균 기온이 올라가면 수영복 판매량이 증가할 것이다.
   - 가설 2: 국민 1인당 GDP가 높아지면 1인당 해외여행비 지출이 증가할 것이다.
   - 가설 3: 소득이 높은 가구일수록 거주하는 주택의 가격이 비쌀 것이다.
   - 가설 4: 공부를 오래 한 학생일수록 시험 성적이 더 높을 것이다.

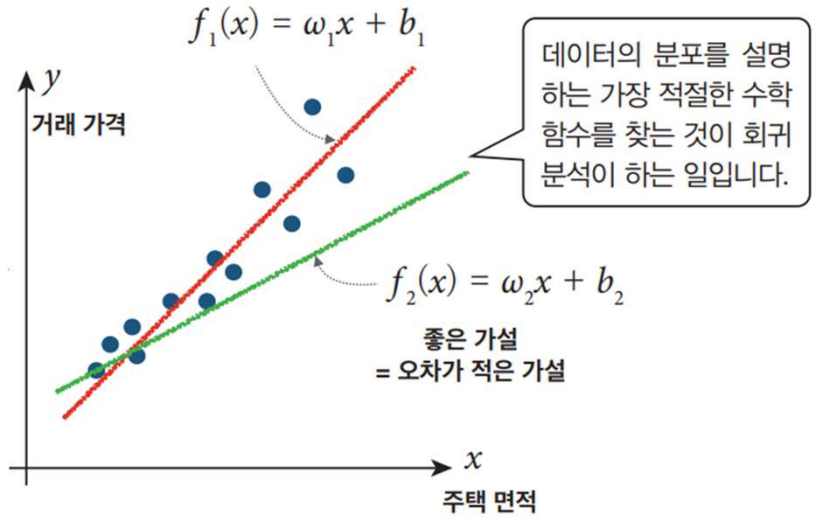
회귀 분석이란 데이터의 분포를 설명하는 좋은 함수를 찾는 것

- 관측된 데이터를 바탕으로 다차원 공간에 존재하는 데이터들을 가장 잘 설명하는 수학 함수를 찾는 것이 바로 회귀 분석이 해야 할 일
  - 이에 대한 다른 표현으로 y = f(x)에서 입력 x와 출력 y를 보면서 함수 f(x)를 예측하는 것을 회귀기법이라고 할 수 있다. '주택 면적'을 x 좌표에 입력하고 이들의 '거래 가격'을 y 좌표에 매핑시킨 후, 이 상관관계를 가장 잘 설명하는 직선을 찾는 문제를 예로 들 수 있다



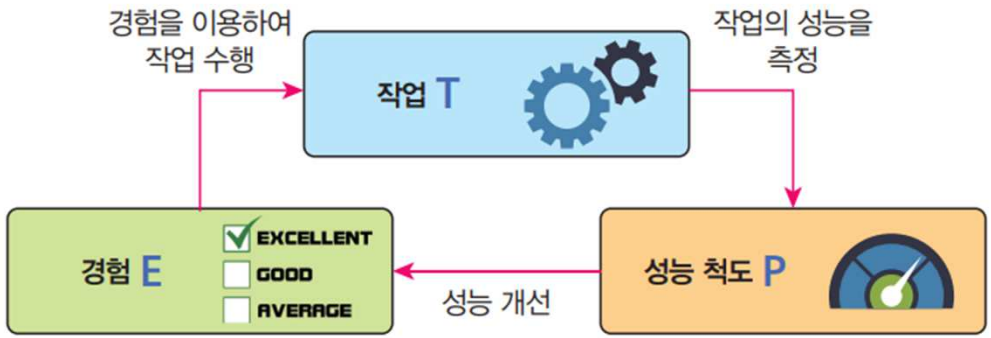
회귀 분석이란 데이터의 분포를 설명하는 좋은 함수를 찾는 것

- 주택 면적과 거래 가격 사이의 상관관계가 일차방정식으로 표현될 수 있는 선형관계라고 가정하면 주택 면적 x와 거래 가격 y의 관계는 그림과 같이 $y = \omega x + b$로 표현될 수 있다
  - 일차 함수는 기울기 ω과 절편 b를 어떻게 정하는가에 따라 $f_1(x) = \omega_1 x + b_1$ 혹은 $f_2(x) = \omega_2 x + b_2$로 표현할 수 있다.
  - 종속변수가 어떤 값을 가질지 예측하는 $f_1(x)$와 $f_2(x)$를 **가설** hypothesis 혹은 **모델** model 이라고 부른다.



회귀 분석이란 데이터의 분포를 설명하는 좋은 함수를 찾는 것

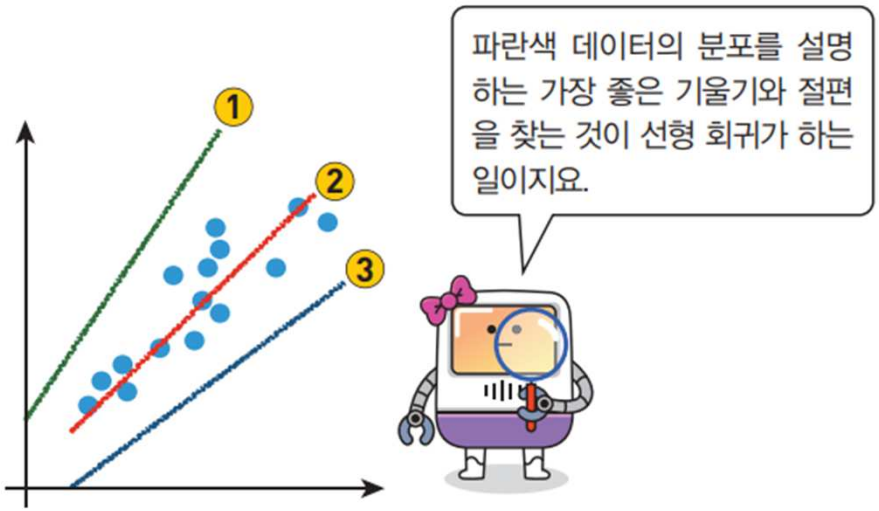
- 회귀 문제에서 y = f(x) 함수의 입력 x에 대응되는 실수 y들이 주어지고 추정한 함수 f()가 만들어내는 오차를 측정하여, 이 오차를 줄이는 방향으로 함수의 계수 ω, b를 최적화하는 과정을 수행한다면 이는 톰 미첼이 정의한 머신러닝으로 볼 수 있다.
  - 작업 T는 독립변수에 대응하는 종속 변수를 추정하는 일이며, 주어진 데이터가 경험 E에 해당한다.
  - 성능 척도 P는 예측한 값 $\hat{y}$과 데이터로 제공되는 목표값 y의 차이가 작을수록 높은 점수를 부여한다.



톰 미첼의 정의에 따른 머신러닝의 요소

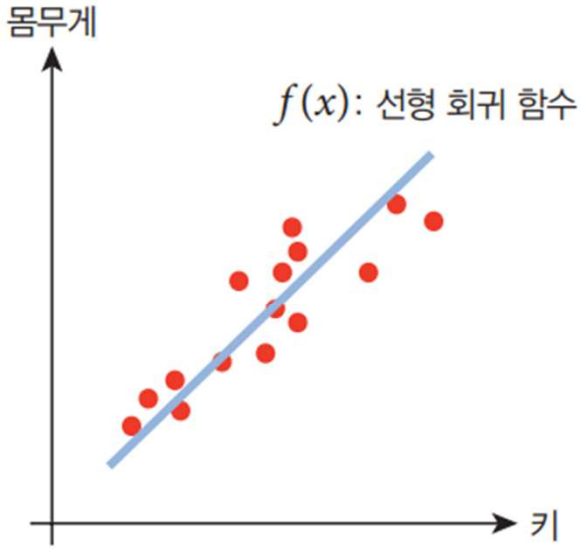
회귀 분석이란 데이터의 분포를 설명하는 좋은 함수를 찾는 것

- 선형 회귀란 데이터를 설명하는 선형적인 관계를 모델링하는 것.
  - 좀 더 구체적으로 이야기하자면 변수 사이의 관계를 설명하는 일차방정식에서 가장 적절한 기울기와 절편 값을 찾는 것으로 볼 수 있다.
  - x 변수는 데이터의 특성이므로 변경할 수 없다.
  - 우리가 제어할 수 있는 값은 기울기와 절편이다. 기울기와 절편의 값에 따라 여러 개의 직선이 있을 수 있다. 기본적으로 **선형 회귀 알고리즘은 데이터 요소에 여러 직선을 맞추어 본 후에 가장 적은 오류를 발생시키는 직선을 반환**한다.
  - 그림을 살펴보면 1 , 2 , 3 중에서 2 가 가장 적은 오류를 발생시키는 직선이다.
  - 이 직선을 어떻게 찾을 수 있을까?

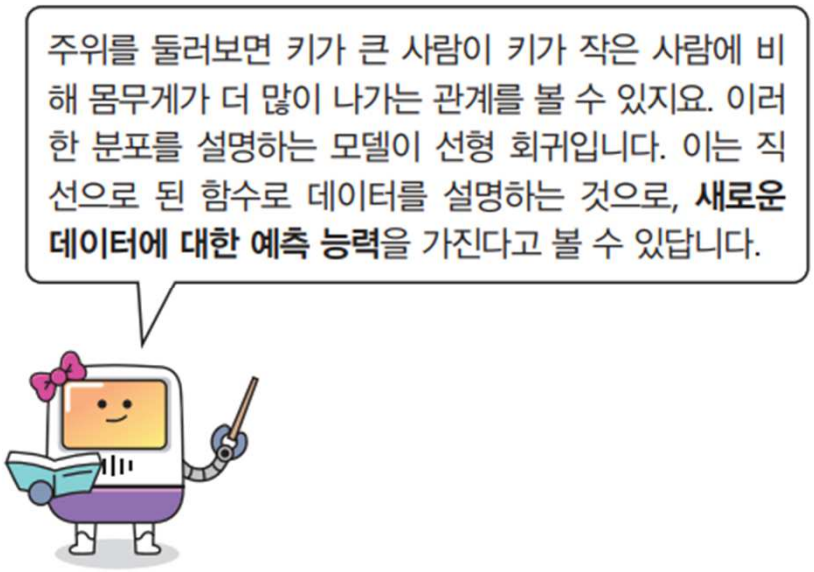


02 오차와 오차의 측정

- 데이터의 분포와 경향성
  - 주택의 면적과 거래 가격이 아닌 사람의 키와 몸무게라는 변수를 이용해 보자.
  - 이 그래프의 가로축은 사람의 키를 나타내며 세로축은 사람의 몸무게를 나타낸다. 샘플의 분포를 시각화한다면 우리는 키가 큰 사람일수록 몸무게가 많이 나가는 경향성을 발견할 수 있을 것이다.

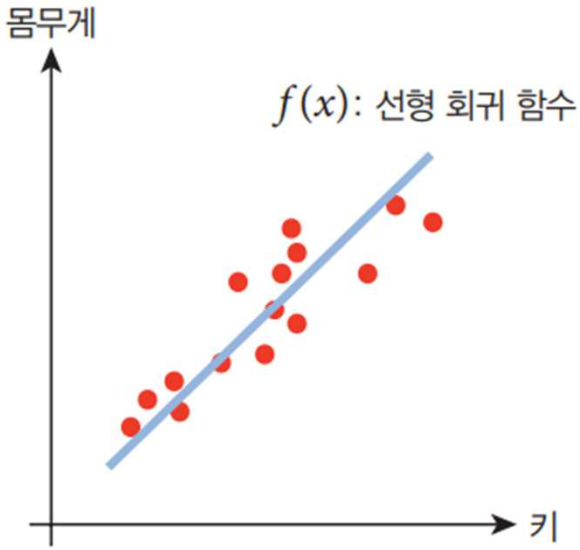


데이터의 분포와 선형 회귀 함수

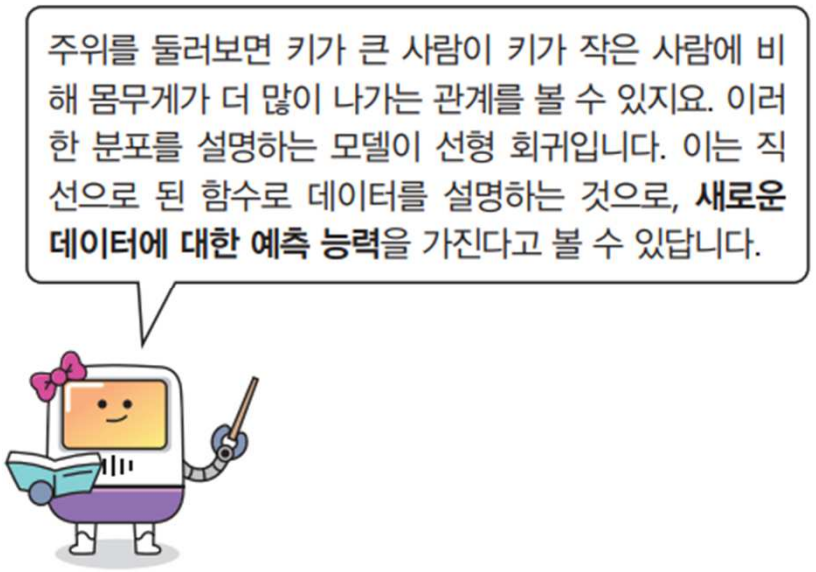


02 오차와 오차의 측정

- 데이터의 분포와 경향성
  - 데이터의 분포를 설명하는 직선을 선형 회귀 직선이라고 한다.
    - 이 직선의 함수 f(x)를 **선형 회귀 함수**라고 한다. 이 함수는 직선(直線)의 형태(形態)를 가지기 때문에 선형(線型, linear)이라는 이름이 붙게 되었다.
    - 선형 회귀는 실제 데이터를 바탕으로 새로운 데이터가 입력되었을 경우의 출력 결과를 예측할 수 있다는 측면에서 의의가 크다.

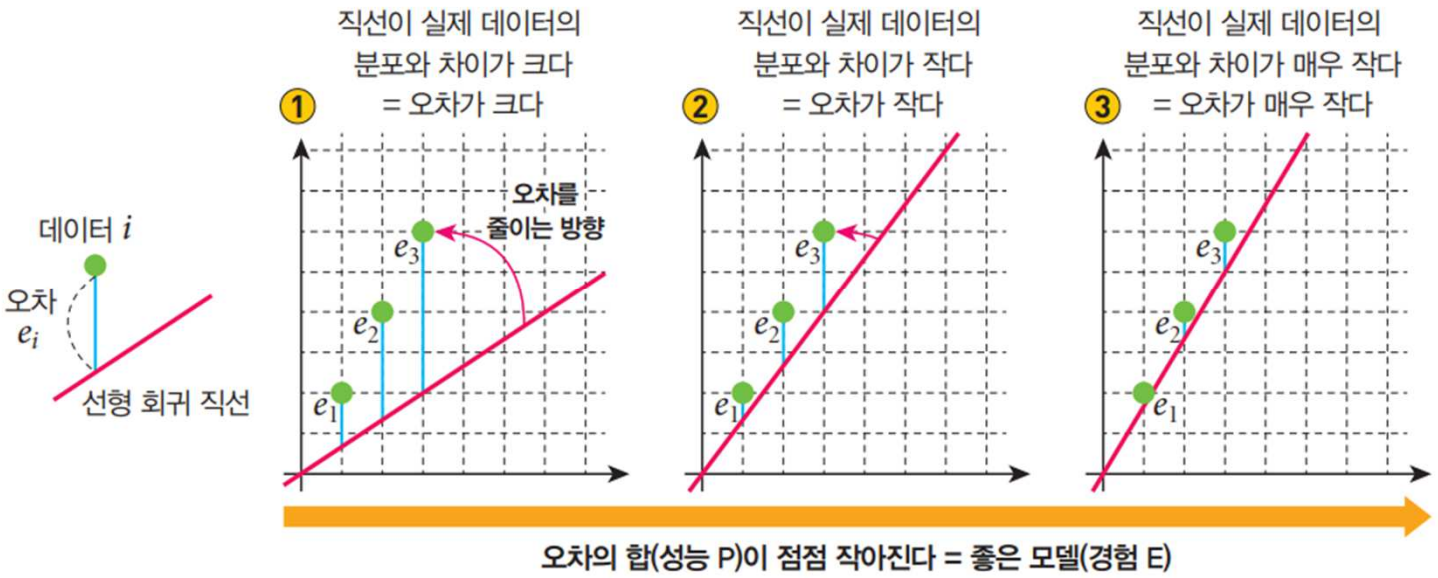


데이터의 분포와 선형 회귀 함수



데이터와 정답의 차이: 오차

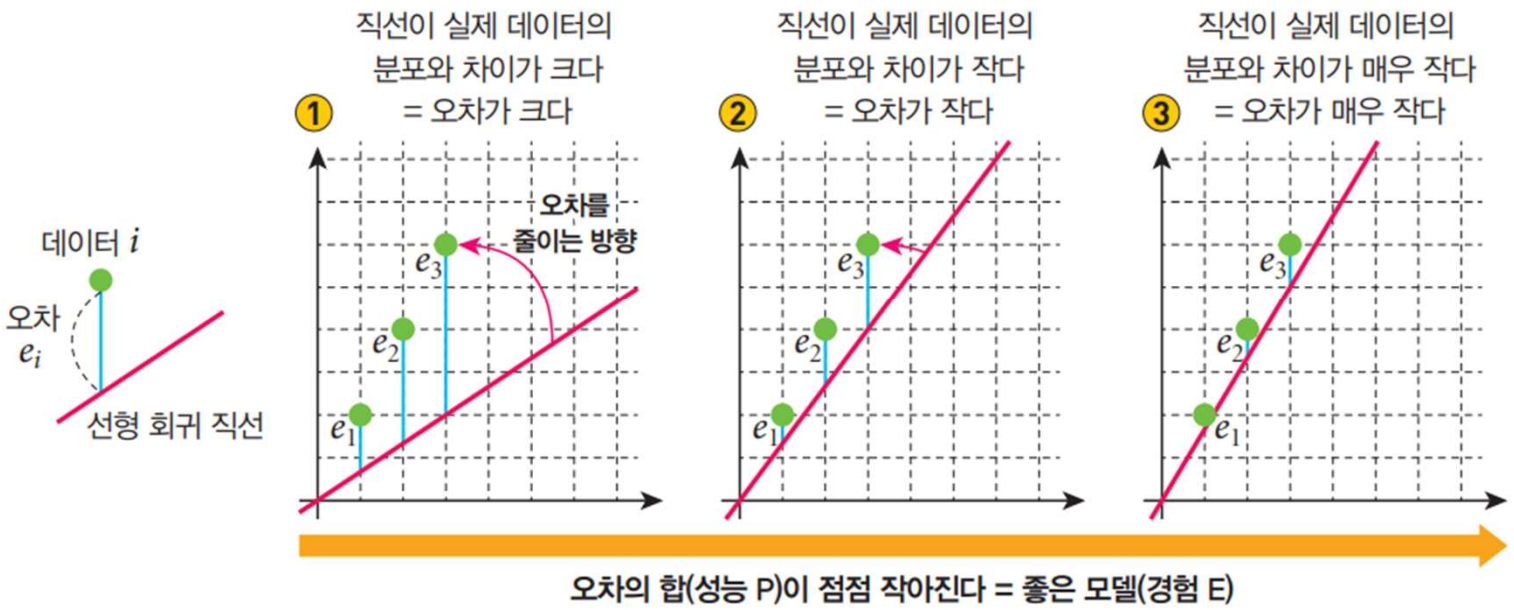
- 문제를 좀 더 단순화시켜 원점을 지나는 직선의 기울기를 찾는 방법으로 만들고 이 문제를 풀어보자.
  - 다음 그림 1 번과 같이 임의의 파라미터 m을 이용하여 직선을 그려보면 이 직선이 데이터의 분포를 제대로 잘 설명하지 못하는 것을 볼 수 있는데, “데이터의 분포를 잘 설명한다”, “데이터의 분포를 잘 설명하지 못한다”와 같은 **주관적인 표현보다는 이를 정량화를 하는 방법이 컴퓨터에는 더 적합한 방법**이다.



데이터와 정답의 차이: 오차

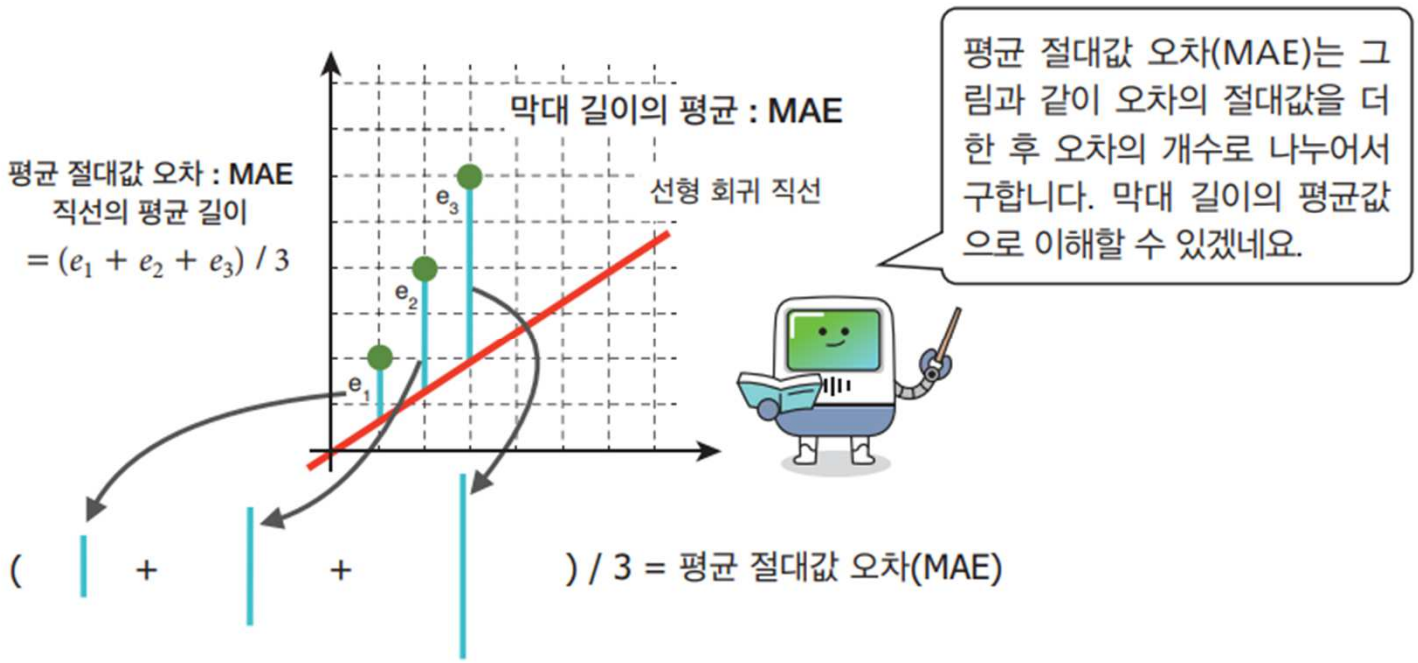
- 1 번 그림의 직선보다 2 번 그림의 직선이 실제 데이터(초록색 점)의 분포를 더 잘 설명하는 직선이며, 3 번 그림의 직선은 2 번의 직선보다 더 나은 직선이다.
  - 그림의 1 번 → 2 번 → 3 번과 같이 데이터를 바탕으로 직선이 오차를 줄이는 방향으로 스스로 고쳐나가도록 하는 것이 머신러닝이 하는 일이다.
  - 이와 같이 오차를 줄여나가는 것을 수학적으로 이야기하면 오차의 합이 점점 작아진다고 할 수 있으며 이는 곧 좋은 모델이 된다는 의미이다.

$e_i = |y_i - \hat{y}_i|$: 오차는 실제 데이터와 추정값의 차이



평균 절대값 오차와 평균 제곱오차

- 평균 절대값 오차
  - 오차의 크기를 어떻게 컴퓨터가 측정할 수 있도록 정량화할 수 있을까 생각해 보자.
  - 다음 그림은 오차값들을 모두 더한 후 3으로 나누어 평균을 구하였다. 따라서 이 값은 $(e_1 + e_2 + e_3)/3$이 된다. 이와 같이 오차의 합을 구하고 그것을 데이터의 개수로 나누어 평균을 구하여 얻은 오차값을 **평균 절대값 오차**Mean Absolute Error: MAE 라고 한다



평균 절대값 오차와 평균 제곱오차

- i번째 데이터의 실제값을 y_i로, 예측값을 $\hat{y}_i$으로, 데이터의 개수를 N으로 나타낼 때 평균 절대값 오차(MAE)는 다음과 같은 식으로 일반화할 수 있다.
  - 선형 회귀에서 예측값을 나타내는 일반적인 표기법 : $\hat{y}$(y 햇으로 읽는다)

$$\mathrm{MAE} = \frac{1}{N}\sum_{i=1}^{N}|y_i - \hat{y}_i|$$

  - 평균 절대값 오차는 직관적이고 계산이 편리한 반면 다음의 문제가 있다.

미분 불가능 지점의 발생

절대값의 사용으로 인해 오차 함수에서 미분이 불가능한 지점이 발생한다는 문제가 있다. 오차 함수에서 미분이 불가능한 구간이 존재할 경우, 점진적으로 최소의 오차값을 찾기 위한 경사하강법을 적용할 수 없게 된다.

평균 절대값 오차와 평균 제곱오차

- 넘파이를 통해서 실제값 y와 추정값 y_hat의 MAE를 구해보자.
  - 넘파이의 np.abs()와 /len(diff)로 구할 수 있으나 사이킷런의 mean_absolute_error도 가능하다.

사이킷런은 머신러닝을 위한 라이브러리로, 뒷장에서 상세하게 다룰 것이다.

```
import numpy as np

# 넘파이를 이용하여 구현한 평균 절대오차
y = np.array([1, 2, 3, 4, 5])                    # 실제 y값
y_hat = np.array([1.2, 2.4, 2.5, 4.6, 5.4])      # 추정한 y값
diff = np.abs(y - y_hat)                         # y_hat과 y 차이의 절대값
e_mae = diff.sum() / len(diff)
print('평균 절대오차 =', e_mae)
```

```
평균 절대오차 = 0.41999999999999993
```

```
print('평균 절대오차 =', np.average(diff))
```

```
평균 절대오차 = 0.41999999999999993
```

```
from sklearn.metrics import mean_absolute_error

# sklearn에서 제공하는 함수를 사용해 보자. 위의 결과와 동일하다.
print('평균 절대오차 =', mean_absolute_error(y, y_hat))
```

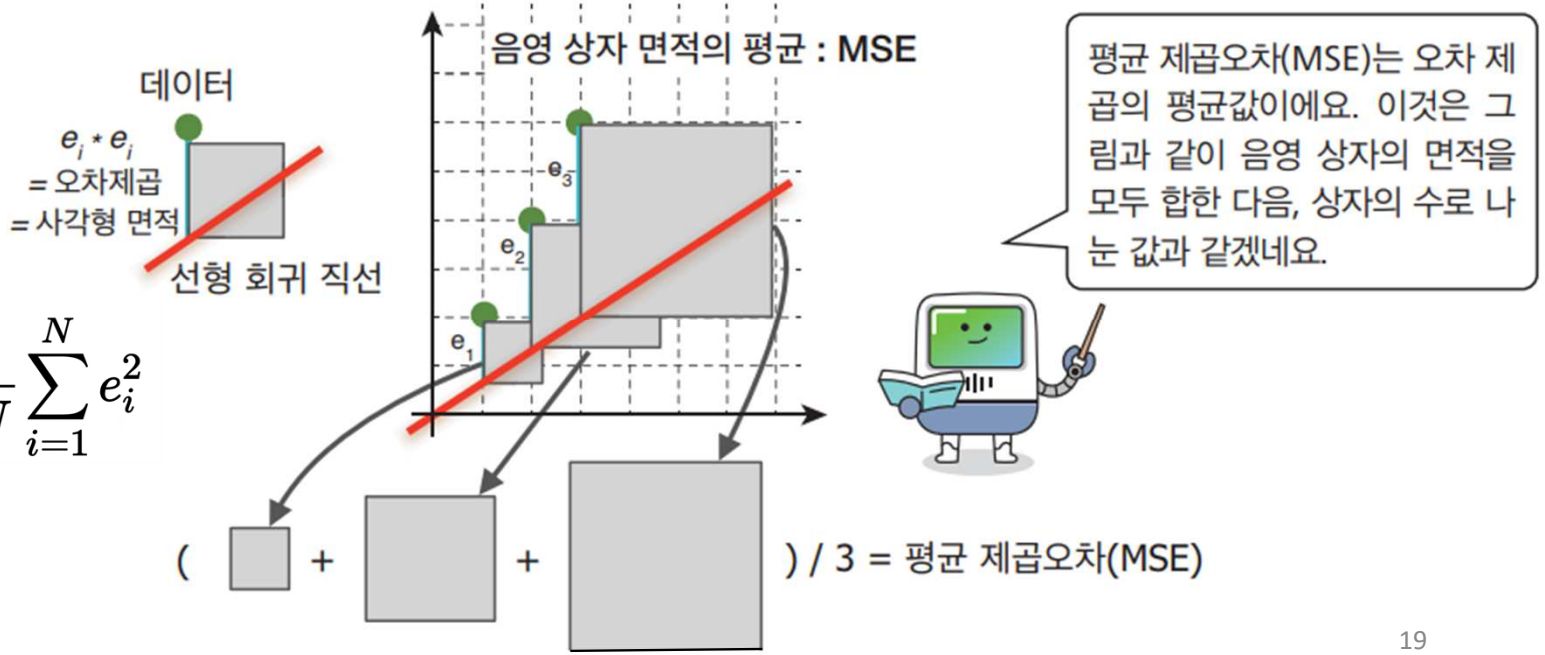
```
평균 절대오차 = 0.41999999999999993
```

평균 절대값 오차와 평균 제곱오차

▪ 평균 제곱 오차Mean Squared Error: MSE

- 평균 절대값 오차가 가진 문제를 해결하기 위하여 e_1이 아닌 이 오차의 제곱인 $e_1 * e_1$을 오차항으로 사용하자.
- 오차의 제곱은 오차를 한 변으로 하는 정사각형의 면적과 같기 때문에 그림과 같이 오차를 이용해서 만든 정사각형 면적의 평균값으로 이해하면 된다.

$$\mathrm{MSE} = \frac{1}{N}\sum_{i=1}^{N}(y_i - \hat{y}_i)^2 = \frac{1}{N}\sum_{i=1}^{N} e_i^2$$



평균 절대값 오차와 평균 제곱오차

- 넘파이를 이용하여 평균 제곱 오차를 구하자
  - 실제값 y와 추정값 y_hat이 있을 경우 사이킷런의 함수를 사용할 수 있다.

```
import numpy as np

# 넘파이를 이용하여 구현한 평균 제곱오차
y = np.array([1, 2, 3, 4, 5])                    # 실제 y값
y_hat = np.array([1.2, 2.4, 2.5, 4.6, 5.4])      # 추정한 y값
diff = (y - y_hat) ** 2                          # y_hat과 y의 차이값의 제곱
e_mse = np.average(diff)
print('평균 제곱오차 =', e_mse)
```

```
평균 제곱오차 = 0.19399999999999995
```

```
from sklearn.metrics import mean_squared_error

# sklearn에서 제공하는 함수를 사용해 보자. 위의 결과와 동일하다.
print('평균 제곱오차 =', mean_squared_error(y, y_hat))
```

```
평균 제곱오차 = 0.19399999999999995
```

평균 절대값 오차와 평균 제곱오차

- 평균 절대값 오차 방식의 계산이 더 간단한데 평균 제곱오차는 왜 필요할까?
  - 평균 제곱오차는 오차에 제곱을 취하기 때문에 오차가 매우 클 경우, 성능 측정 함수 P의 값이 급격하게 커진다.
  - 달리 말하면 선형 회귀 모델이 예측한 직선이 실제 데이터와 차이가 많이 나면, 이 모델의 성능이 나쁘다는 것을 알리도록 모델을 만드는 것이다.
  - 회귀 모델의 오차는 일반적으로 **평균 제곱오차**를 널리 사용한다.
  - 오차의 제곱은 오차 직선을 한 변으로 하는 정사각형의 면적과 같은데 이를 구하면 항상 양수가 되며, 선형 회귀 직선이 데이터의 분포를 가장 잘 설명하는 경우, MSE 값은 최소값이 된다.
  - 이와 같이 데이터의 분포를 가장 잘 설명하는 선형 회귀 직선을 구하는 과정을 **최적화** optimization 라고 한다.
    - 이 최적화 과정 = 머신러닝의 학습과정이다.

평균 절대값 오차와 평균 제곱오차

도전 문제 4.2: 평균 절대오차와 평균 제곱오차 구하기

상 중 하



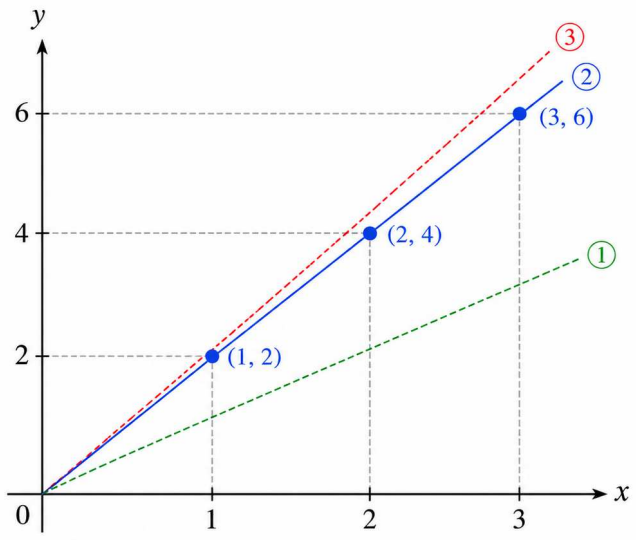
1. 정답값 벡터가 [2, 4, 6, 8, 10]일 때, 모델 A의 추정값이 [2.1, 4.3, 5.5, 7.9, 10.1]로 구해졌다. 이 모델 A와 정답 사이의 평균 절대오차와 평균 제곱오차를 넘파이를 이용하여 구한 다음, 다음과 같이 출력하여라.

```
모델 A의 평균 절대오차 = 0.21999999999999983
모델 A의 평균 제곱오차 = 0.07399999999999997
```

2. 위의 데이터에 대하여 평균 절대오차와 평균 제곱오차를 다시 구하여라. 이를 위하여 사이킷런의 함수 `mean_absolute_error()`와 `mean_squared_error()`를 사용하여라.

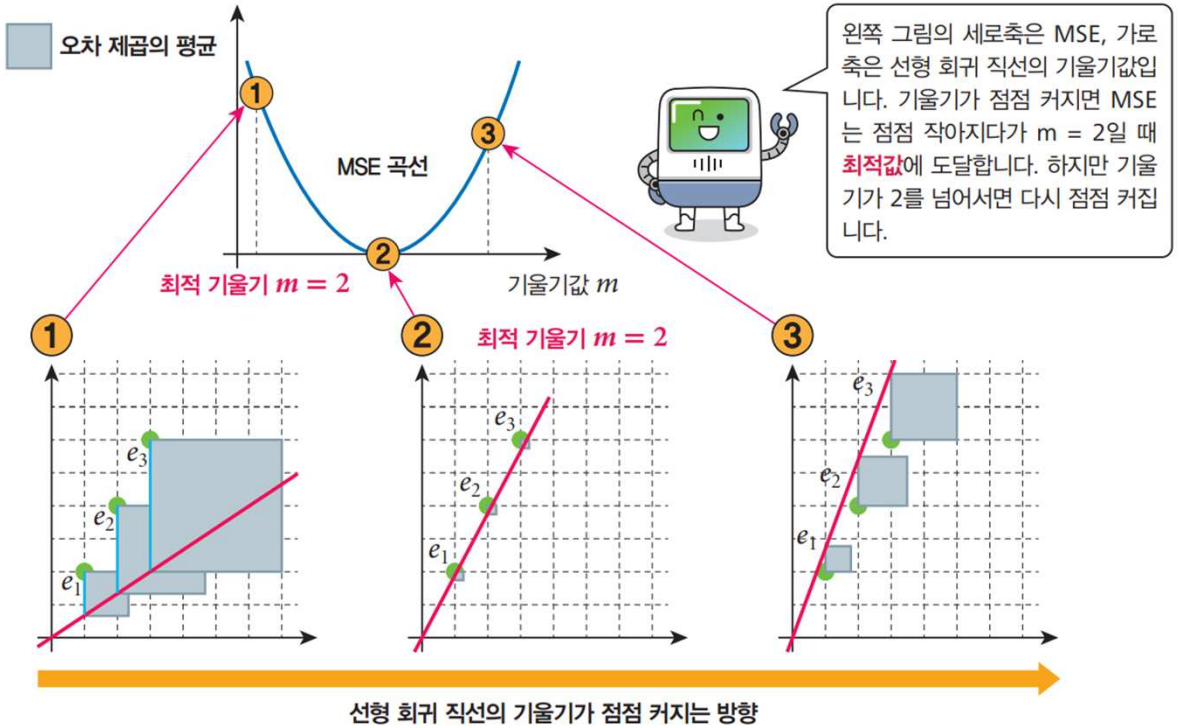
선형 회귀 직선 구하기 예시

- 그림과 같이 (1, 2),(2, 4), (3, 6)의 3개 데이터가 2차원 공간에 분포하는 경우를 가정하자. 이 데이터를 2차원 평면에서 표현하기 위하여 1, 2, 3을 x 축에 배치하고 이에 대응하는 값을 y 축의 값으로 두었다
  - 이 간단한 데이터의 분포를 설명하는 선형방정식 중에서 가장 좋은 선형방정식을 구하기 위해서 다음과 같은 그림을 살펴보자. 그림의 1 , 2 , 3 은 각각 다음을 의미한다 (이 데이터의 경우 선형 회귀 직선의 최적 기울기값은 2가 될 것이다).
  - 1 : 직선의 기울기 m이 2보다 작은 경우
  - 2 : 직선의 기울기 m이 2인 경우(최적 기울기)
  - 3 : 직선의 기울기 m이 2보다 큰 경우



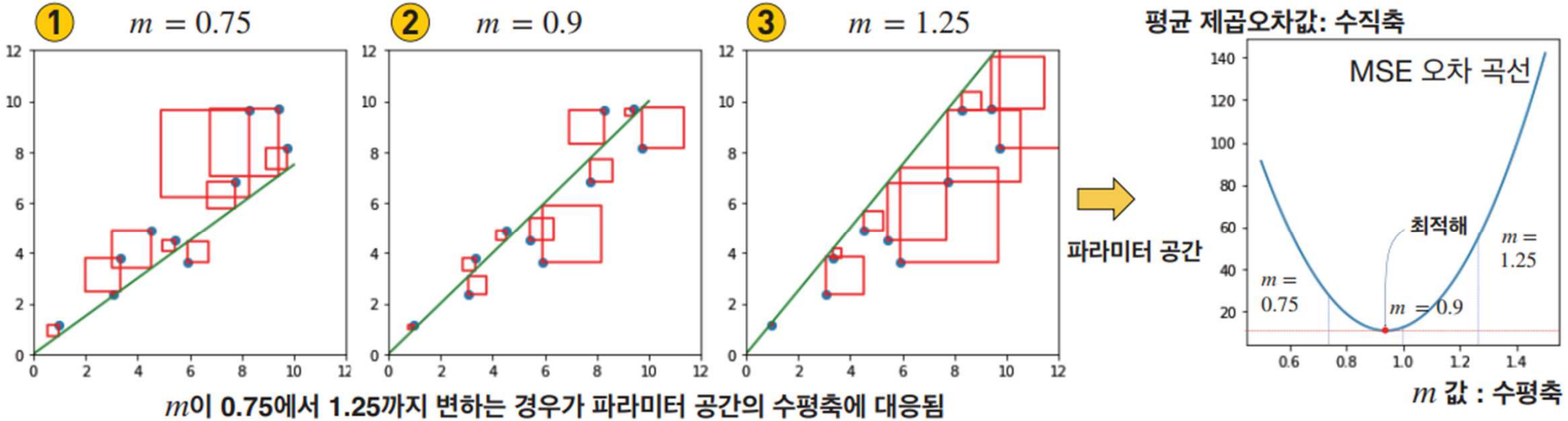
선형 회귀 직선 구하기 예시

- MSE 곡선의 가로축은 선형 회귀 직선의 기울기값 m이며 세로축은 MSE 값을 나타낸다.
  - 이 그림에서 세로축은 전체 오차의 크기이므로 곡선에서 오차의 크기가 가장 작은 2번 위치가 모델의 성능이 가장 좋은 상태이다.
  - 이는 아래쪽 그래프의 최적 기울기를 가질 때(즉, m = 2일 때)와 동일하다.



선형 회귀 직선 구하기 예시

- 평균 제곱오차를 사용하는 이유?
  - **오차 합 곡면의 기울기를 따라 내려가 최소 오차에 접근하기 위해서**이다.
  - 양수 오차가 많으면 그 합이 무한히 커질 수도 있고, 음수 오차가 많으면 무한히 작은 값을 가질 수도 있다.
  - 하지만, **오차를 제곱하면 볼록한 그릇 모양의 곡면**을 만들 수 있다.
  - **가장 좋은 파라미터는 그릇 모양의 가장 아래쪽 구간임.**



m이 0.75에서 1.25까지 변하는 경우가 파라미터 공간의 수평축에 대응됨
이때, 평균 제곱오차는 사각형 면적의 평균값으로 파라미터 공간의 수직축에 대응됨

오차 곡선의 최적값을 구하는 기울기와 미분

- 그림의 곡선을 **오차 곡선**이라고 하는데, 다차원 공간일 경우 **오차 곡면**이 된다.
  - 오차 곡선에서 최소값(혹은 최적해)을 구하기 위하여 오차 곡선 혹은 곡면의 기울기를 따라 내려가며 해를 구하는 **경사하강법** gradient descent method 이라는 방법으로 해를 찾아볼 것이다

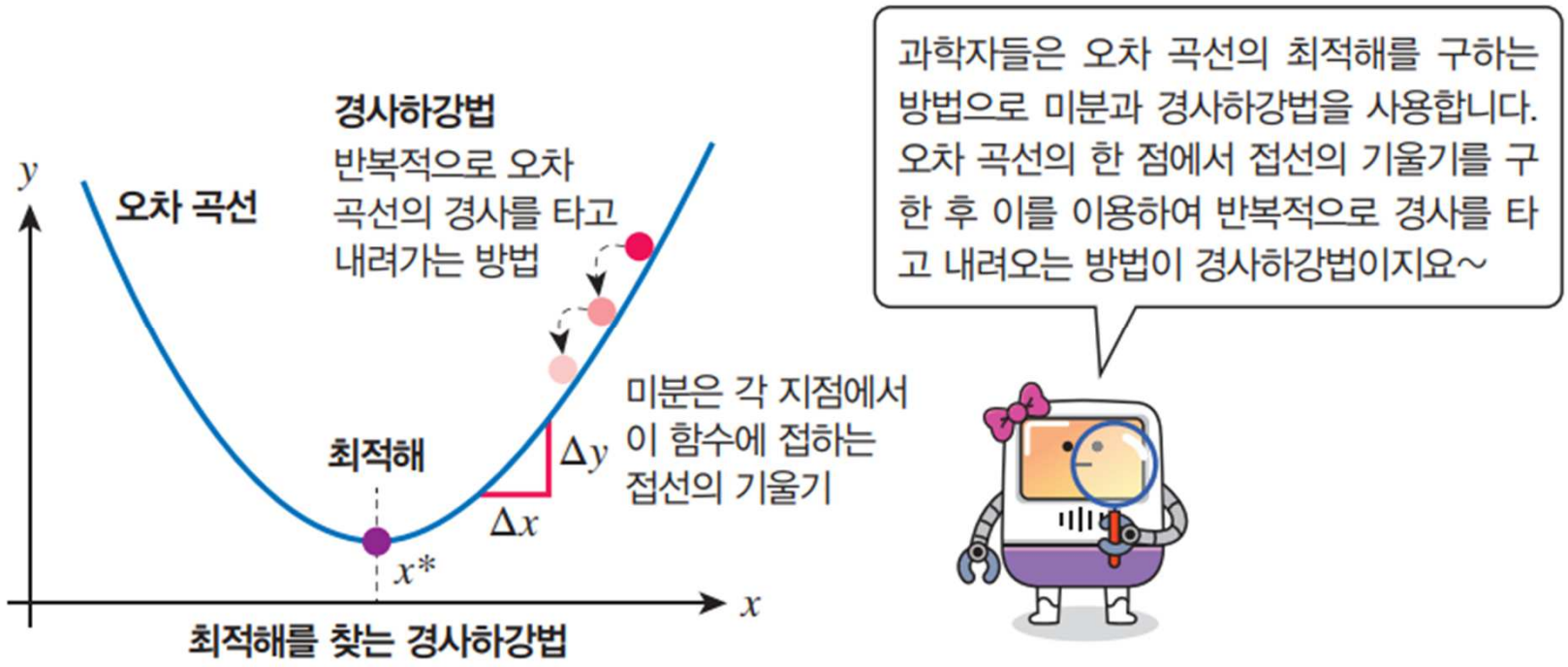


오차 곡선의 최적값을 구하는 기울기와 미분

- 그림에서 오차 곡선(MSE 곡선)의 가장 낮은 부분이 최적값이라고 표시되어 있다.
  - 이 그림과 같이 처음에 시작할 때 임의의 기울기값을 사용한 위치는 시작점의 하나가 될 수 있을 것이다.
  - 우리가 해야 할 작업 T는 오차 곡선의 최적값 구하기이며 이것은 성능 척도 P로 측정할 수 있다.
  - 이제 곡선에서 한 점의 기울기를 구할 수 있다면 우리는 이 기울기를 구한 후 곡선이 어느 정도 가파른지 혹은 양의 방향(+)의 기울기인지, 음의 방향(–)의 기울기인지도 알 수 있을 것이다.
  - 이렇게 기울기와 부호를 구한 후 곡선의 아래쪽으로 향하도록 다음 값을 조정하여 다시 한번 오차를 구하고, 곡면의 기울기를 구하는 작업을 반복적으로 수행하면 된다.
  - 이것이 **경험 E**에 해당한다.

오차 곡선의 최적값을 구하는 기울기와 미분

- 경사 하강법은 곡선의 한 점에서 접선의 기울기를 구한 후 이를 이용하여 경사를 타고 내려오는 방법이다.
  - 경사를 타고 오차값이 더 작은 쪽으로 내려오는 작업을 반복적으로 할 수 있다면 우리는 최적값에 도달하는 것도 가능할 것이다.



오차 곡선의 최적값을 구하는 기울기와 미분

도전 문제 4.3: 평균 절대오차와 평균 제곱오차를 구해 보자

상 중 하



1. 다음과 같이 정답값, 모델 A의 예측값, 모델 B의 예측값이 있다. 모델 A와 모델 B의 평균 절대오차(MAE), 평균 제곱오차(MSE)를 각각 구하여라.

정답값	모델 A	모델 B
1	0.9	0.4
2	1.3	1.9
3	3.3	3.4
4	3.8	4.2

2. 위의 오차를 구하는 기능을 넘파이를 이용하여 구현하여라.

3. 위의 오차를 구하는 기능을 사이킷런의 metric 서브 모듈에 있는 함수를 사용하여 구현하여라.

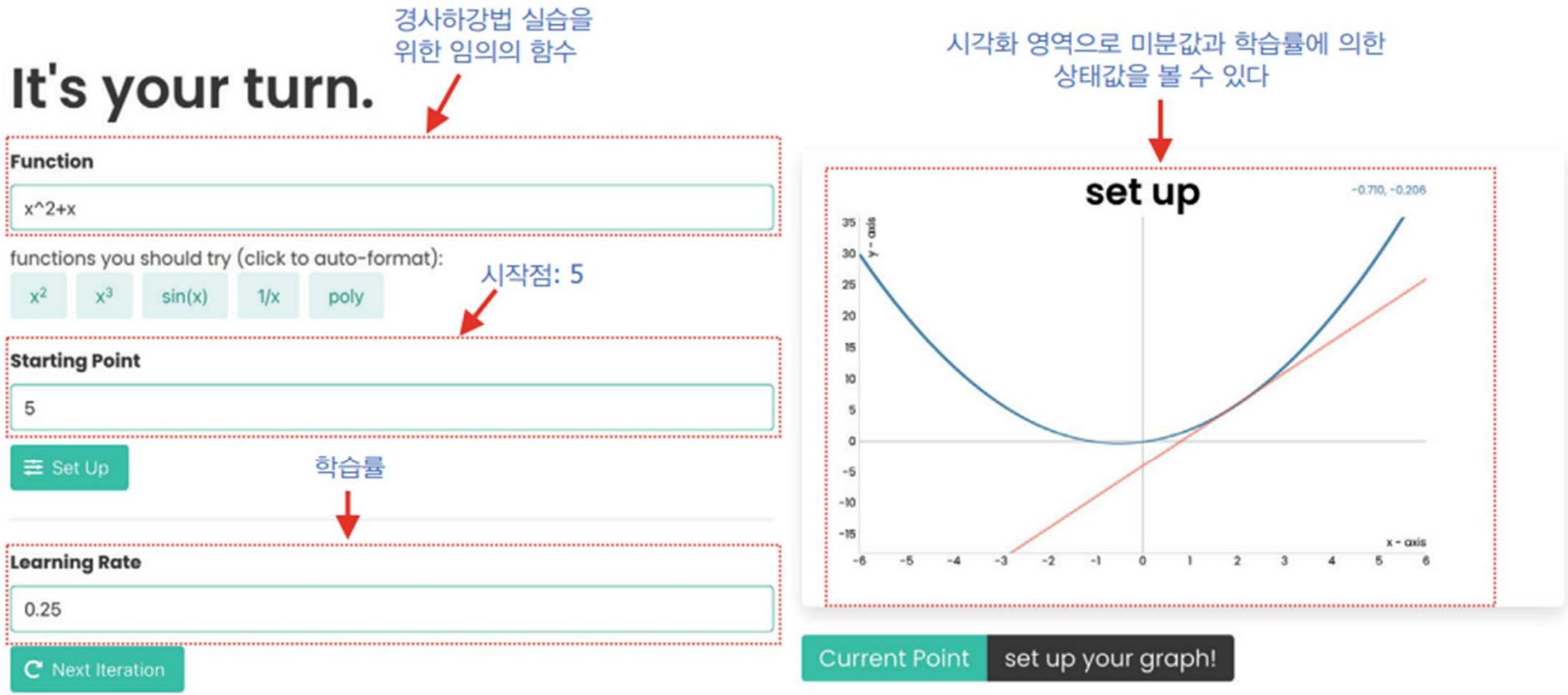
4. 위의 두 모델 중에서 어느 모델이 더 나은 모델이라고 주장할 수 있는가? 그 근거를 제시하여라.

03 경사하강법을 알아보는 실습

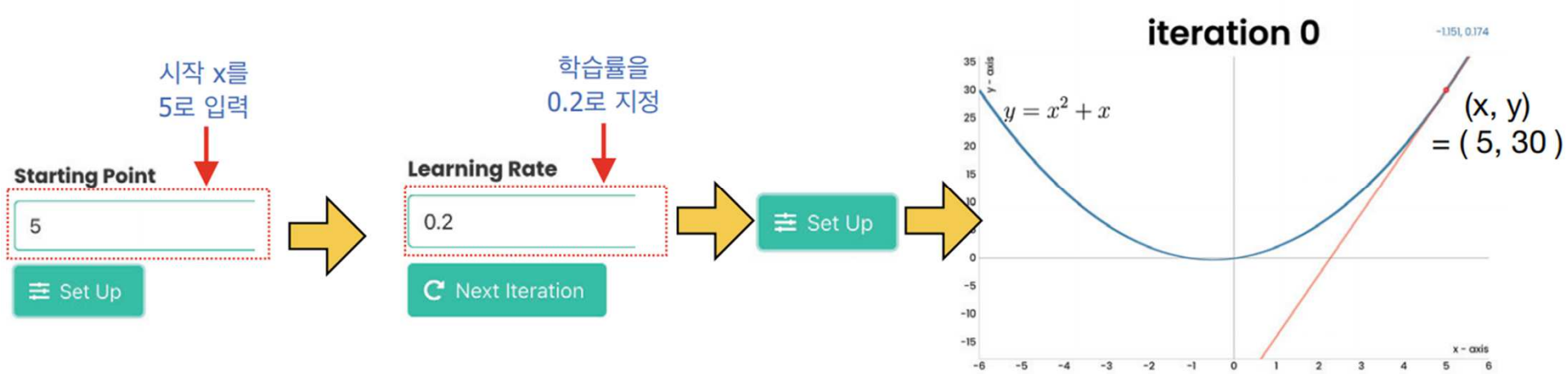
- 경사하강법은 대표적인 최적화 기법이다.
  - 임의의 곡선이나 곡면이 있을 경우 이것의 극소값을 찾기 위해서는 어떻게 해야 할까? 곡선의 방정식을 풀어서 그 해를 구하는 방법도 있겠지만 이 방식은 컴퓨터에서는 적합하지 않다.
    - 그 이유는 곡선이나 곡면의 함수들이 닫힌 형태가 아니거나 그 형태가 너무나 복잡해서 미분계수와 그 해를 구하는 것이 어려울 때가 많기 때문이다.
- 대신 컴퓨터가 잘하는 반복적인 작업능력을 활용하는 것이 어떨까?
  - 다음 웹 사이트에 접속하여 이를 실습해 보자.

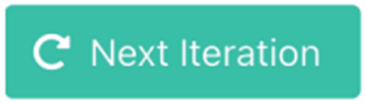
https://uclaacm.github.io/gradient-descent-visualiser/#playground

▪ 단계 1: 경사하강법을 실습하는 방법 : 화면의 구성



- **단계 2** : 다음 순서대로 입력해 보자.

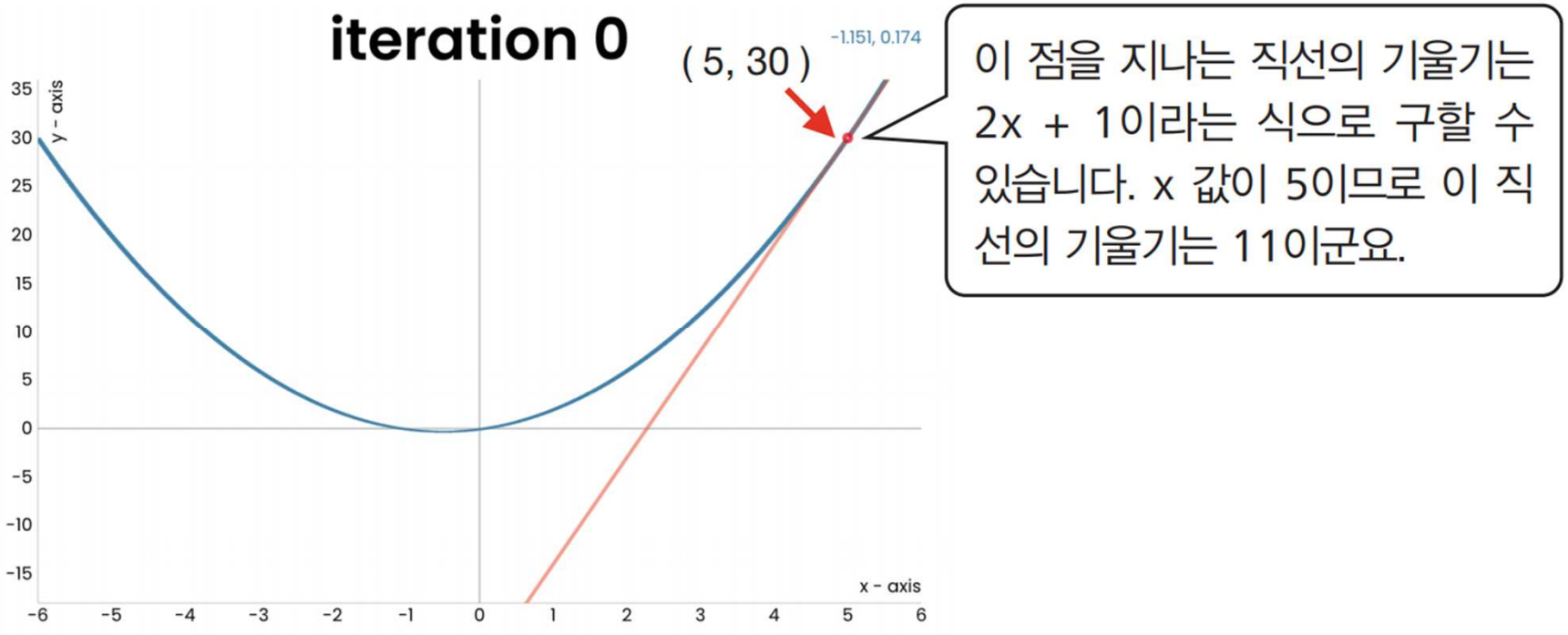


- **단계 3** : [Next Iteration] 버튼을 클릭 

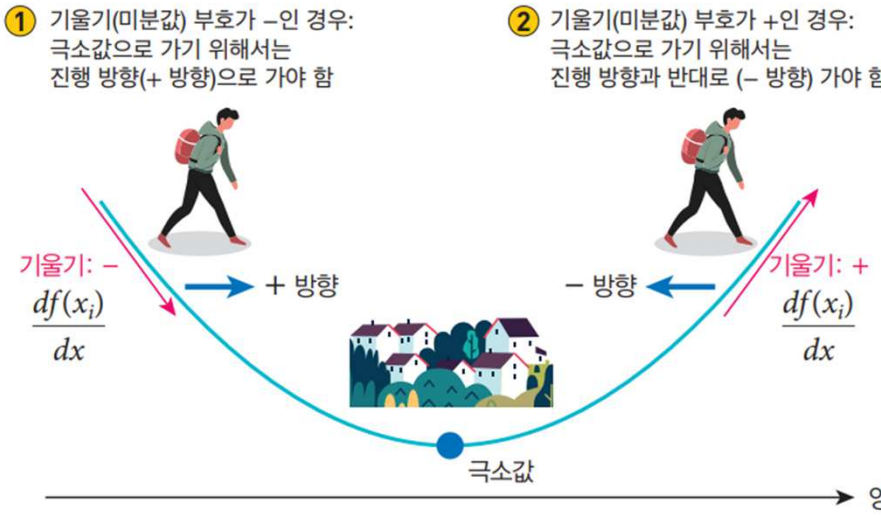
  - 1차 미분 함수 f'(x)는 2x + 1인데, 미분의 의미는 임의의 구간에서의 순간적인 변화량이라는 것을 되새기도록 하자.
  - 이제 x = 5일 때의 미분값을 구해보면 2 * 5 + 1 = 11이 되는 것을 알 수 있다.

03 경사하강법을 알아보는 실습

- 미분값의 부호가 양수가 된다는 것은 곧 기울기가 양의 값이 된다는 의미이며, 이 곡선의 극소값이 5보다 작은 값이라는 의미이다.



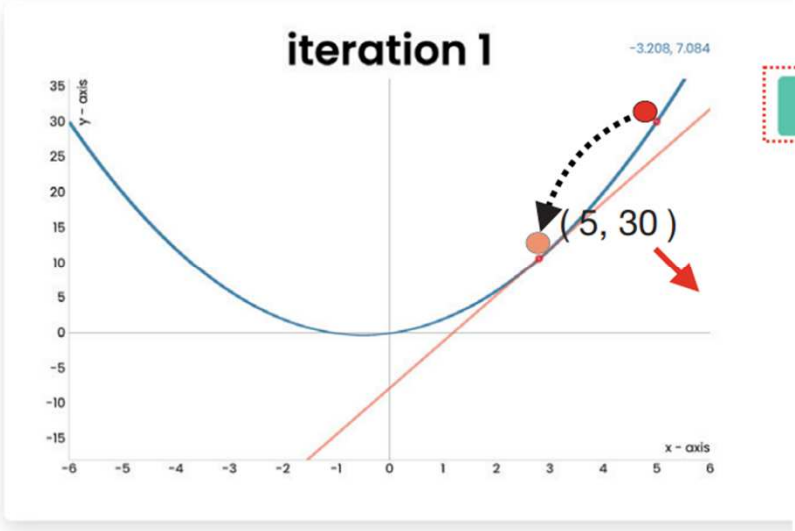
- 경사하강법의 핵심 원리
  - 우리가 깜깜한 산길을 걷다가 길을 잃었다고 생각해 보자. 산 아래쪽에 있는 마을(여기에서는 극소값)로 가야 한다.
  - 이때, 우리가 향하고 있는 산길이 내리막 방향, 즉 기울기(미분값) 부호가 –인 경우라면 어떻게 해야 할까? 아마도 그림 1 과 같이 진행 방향(+ 방향)으로 계속해서 가야만 할 것이다.
  - 반면 우리가 향하고 있는 산길이 오르막 방향, 즉 기울기(미분값)부호가 +인 경우라면 어떤 선택을 해야 할까? 아마도 그림 2 와 같이 진행 방향의 반대 방향(– 방향)으로 가야만 할 것

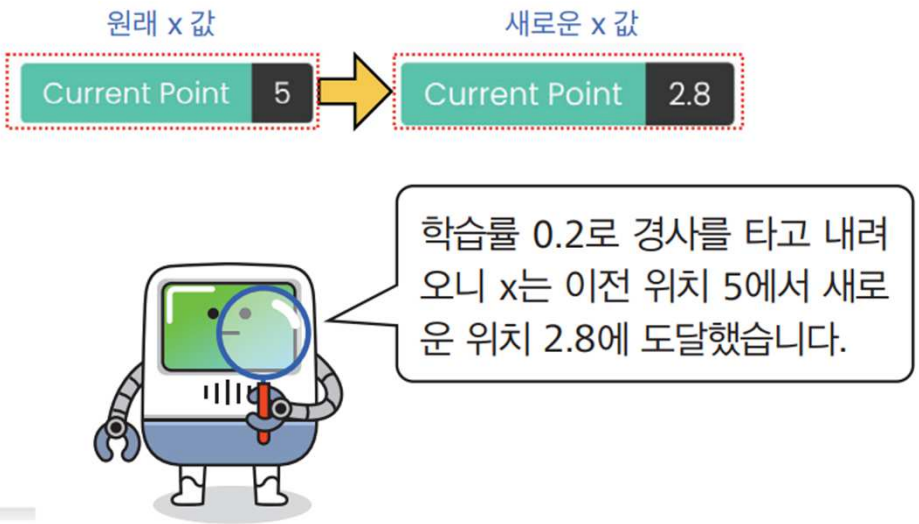


- **단계 4** : 다음은 Next Iteration을 클릭한 결과이다.
  - 새로운 위치값은 다음과 같은 수식으로 얻을 수 있다. 즉 x의 새로운 위치값 xi+1은 이전 단계의 위치값 xi로부터 구하는 식이다.
  - 이 식에서 α는 학습률이며 이 식의 앞에 음의 부호(-)가 붙는 이유는 앞의 그림과 같이 미분의 부호가 +인 경우, 진행 방향의 반대 방향으로 가야함.

$$x_{i+1} = x_i - \alpha \frac{df(x_i)}{dx}$$

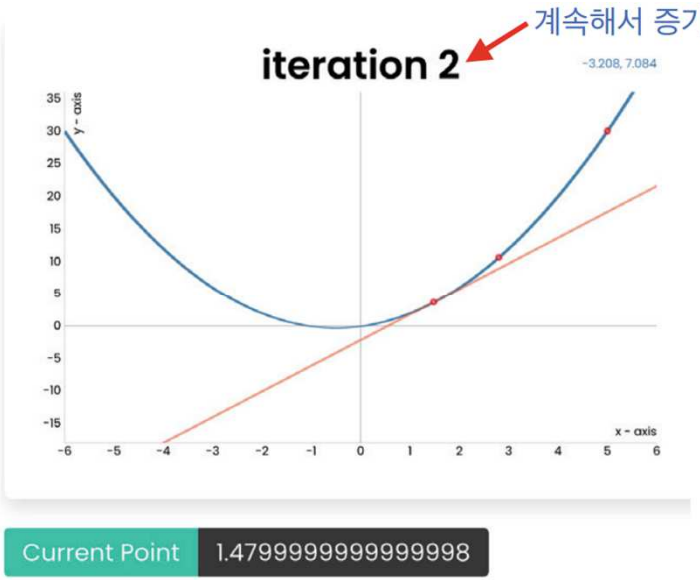
$$\begin{aligned} x_{i+1} &= x_i - \alpha \frac{df(x_i)}{dx} \\ &= 5 - 0.2 \times 11 \\ &= 2.8 \end{aligned}$$

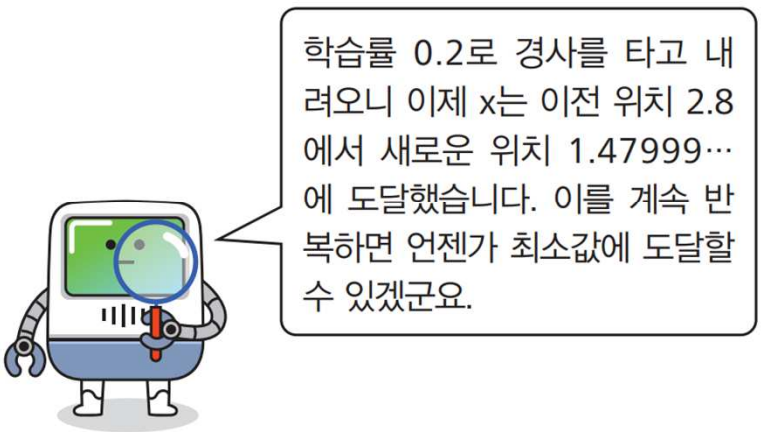




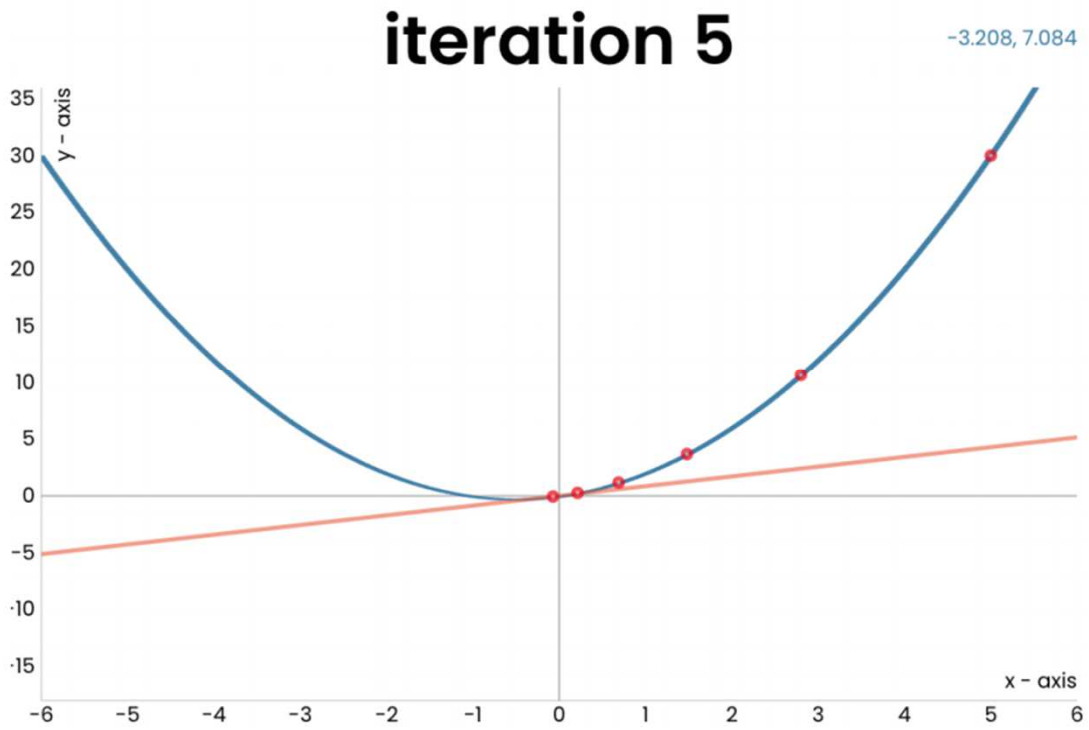
- **단계 5** : 다음 단계로
  - x_{i+1}이 이제는 2.8이므로 x 값이 2.8일 때 직선의 기울기를 구할 수 있다.
  - 이 기울기 f'(x)는 2x + 1에 의해 6.6이 된다.
  - 다시 한번 Next Iteration을 클릭해서 x_{i+2}를 구해 보자.
  - x_{i+2}를 다음 식으로 전개해서 풀어보면 1.48이 된다.
    - 실수 연산의 오차로 인해 출력은 1.4799999999998로 나타난다.

$$x_{i+2} = x_i - \alpha \frac{df(x_{i+1})}{dx}$$
$$= 2.8 - 0.2 \times 6.6$$
$$= 1.48$$





- **단계 6** : 다음 단계로..
  - iteration 5가 되면 이전에 비해 최소값에 매우 가깝게 도달한 것을 확인할 수 있다
  - 이 프로그램은 작업 T와 성능 척도 P에 대해 경험 E로부터 학습을 하여 최적값에 도달하므로 경사하강법은 머신러닝 기법의 일종으로 간주할 수 있다.



- 다음 표를 채워보며 경사하강법을 익혀보자

단계(i)	x	$f(x) = x^2 + x$	$\frac{df(x_i)}{dx} = 2x + 1$	α	$x_{next} = x - \alpha\frac{df(x_i)}{dx}$
0	5	30	11	0.2	5 − (0.2 * 11) = 2.8
1	2.8				
2					
3					

- 이제 시작 위치가 -5일 때의 경우에 대하여 최적화 수행이 되는가 확인하자

도전 문제 4.4: 학습률에 따른 최적화 과정

상 중 하



1. 이 웹사이트에서 시작 위치를 −5로, 학습률(α)을 0.2로 두고 위의 과정을 반복해 보자. 시작 위치가 −5일 경우 미분값의 부호는 무엇인가?

2. 학습을 5번 반복시켜 보자. 각 단계별 과정을 다음의 표에 있는 값을 채우면서 살펴보자.

단계(i)	x	$f(x) = x^2 + x$	$\frac{df(x)}{dx} = 2x + 1$	α	$x_{next} = x - \alpha\frac{df(x)}{dx}$
0	−5	20	−9	0.2	−5 − (0.2 * −9) = −3.2
1	−3.2				
2					
3					
4					

04 사이킷런을 이용한 선형 회귀

- 여러 가지 머신러닝 라이브러리들
  - 파이썬에서 사용할 수 있는 머신러닝과 딥러닝 라이브러리들은 매우 많다.
  - 이들 중에서 가장 유명한 라이브러리 중의 하나가 **사이킷런** scikit-learn 이다.
  - 이 라이브러리는 2007년도 구글 하계 코드 프로젝트 모임에 참여한 몇몇 개발자들이 중심이 되어 시작되었다.
    - 현재, 이 프로젝트는 머신러닝을 위한 무료 라이브러리 배포라는 큰 목적을 위하여 scikit-learn.org라는 웹사이트를 통해서 오픈 소스 개발 방식으로 개발이 이루어지고 있다.



04 사이킷런을 이용한 선형 회귀

- 머신러닝의 일반적인 단계
  ❶ 특성과 레이블(레이블은 지도학습에 대하여 필요함)로 이루어진 데이터의 준비
  ❷ 데이터를 바탕으로 동작이 결정되는 모델의 결정,
  ❸ 모델을 위한 적절한 하이퍼파라미터 hyperparameter 설정하기,
  ❹ 데이터를 바탕으로 이루어지는 모델의 학습,
  ❺ 학습 결과에 대한 검증으로 나누어 볼 수 있다.
- 사이킷런은 지도학습, 비지도학습을 위한 다양한 데이터와 모델을 제공하며, 이 모델을 위한 시각화 도구, 교차 검증 도구들까지 매우 광범위한 기능을 제공한다.



간단한 데이터로 선형 회귀를 수행해 보자

- 동민이네 모둠에 모두 6명의 학생이 있다고 가정하자.
  - 우리는 이들 중 동민이를 제외한 5명의 학생들에 대해 키와 몸무게를 측정하고 이를 바탕으로 선형 회귀 모델을 만들고자 한다.
  - 5명 학생들의 키는 163, 179, 166, 169, 170cm로 나타났으며, 이들의 몸무게는 각각 54, 63, 57, 56, 58kg으로 나타났다. 일반적으로 키와 몸무게는 상관관계가 있기 때문에 이 상관관계를 선형방정식으로 만들 수 있다.
  - 이 선형방정식이 동민이네 모둠 학생들의 키-몸무게 데이터의 분포를 잘 설명하는 식이라고 한다면, 이 샘플에 포함되지 않은 학생의 키가 167cm일 경우에 몸무게를 유추할 수 있을 것이다



간단한 데이터로 선형 회귀를 수행해 보자

- 사이킷런 라이브러리를 사용하여 회귀 함수를 구현하는 방법을 살펴보자.
  - 사이킷런을 코드에 가져오기 위해서는 sklearn이라는 이름으로 가져와야 한다.
  - 선형 회귀를 위해 가장 먼저 해야 할 작업은 사이킷런 라이브러리와 넘파이를 코드에 import시키는 일이다.
  - 선형 회귀를 구현하기 위해 다음과 같이 선형 모델 linear_model을 import한 뒤에 LinearRegression()생성자를 통해 선형 회귀 모델을 생성한다.
  - 이 선형 회귀 모델을 참조하는 변수는 regr로 지정하자

```
import numpy as np
from sklearn import linear_model  # scikit-learn 모듈을 가져온다

# 선형 회귀 모델을 생성한다
regr = linear_model.LinearRegression()
```

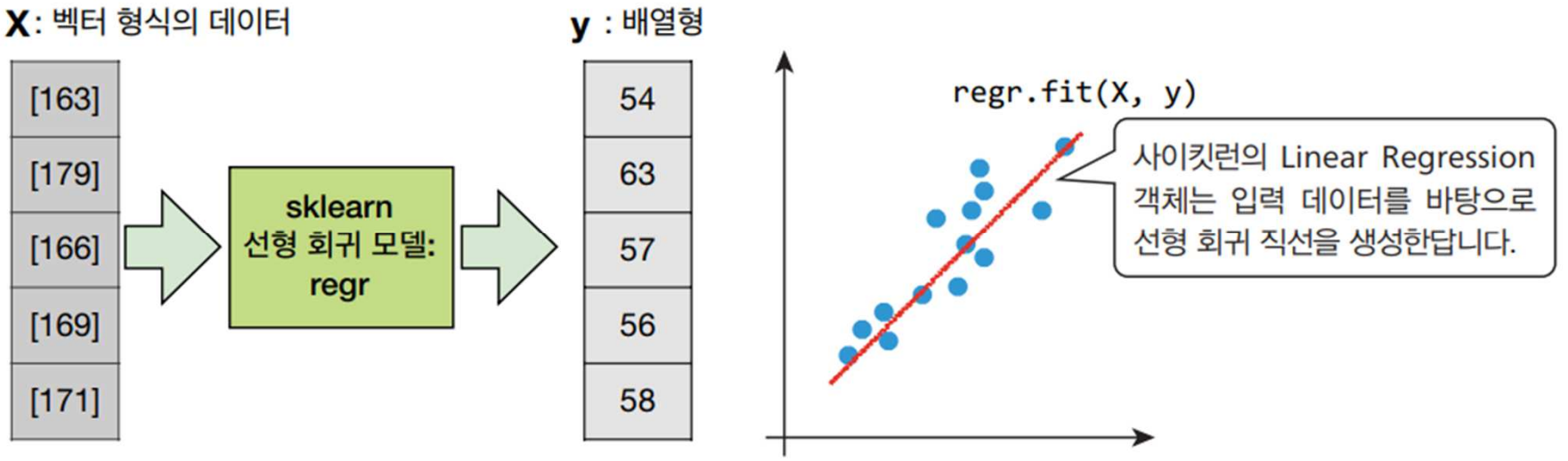
간단한 데이터로 선형 회귀를 수행해 보자

- 선형 회귀를 위한 입력 데이터 집합 X를 만들도록 하자.
  - 입력 데이터는 X = [[163], [179], [166], [169], [171]]과 같은 2차원 리스트로 만들도록 한다.(2차원 리스트이므로 일반적으로 대문자 X를 사용)
  - 주의 : 반드시 2차원 리스트를 입력으로 사용해야만 한다.
  - 다음으로 정답에 해당하는 y 변수를 [54, 63, 57, 56, 58]과 같이 초기화하도록 하자(스칼라 값의 모임이므로 일반적으로 소문자를 사용)
  - regr.fit(X, y)와 같이 선형 회귀 모델에 입력과 출력을 지정하면 최적의 선형 회귀 함수를 찾는 기능을 한다.

```
X = [[163], [179], [166], [169], [171]]
y = [54, 63, 57, 56, 58]
regr.fit(X, y)
```

간단한 데이터로 선형 회귀를 수행해 보자

- regr의 fit() 메소드에서 사용하는 학습 데이터는 반드시 2차원 배열이어야 한다
  - 그 이유는 사이킷런의 LinearRegression() 모델은 다중 회귀 분석을 실시하기 위해서 설계되었기 때문이다. 이때문에 X의 각 항목을 스칼라값이 아닌, 다수의 독립변수를 포함하는 벡터로 간주한다.
  - 입력의 차원이 1차원인 경우에도 163이 아닌 [163]과 같은 배열 형태로 만들어야 함.
  - 그러나 y 값은 목표값으로 1차원 배열형 자료를 사용함.



간단한 데이터로 선형 회귀를 수행해 보자

- 사이킷런은 관례적으로 벡터 값을 대문자로 표기하고 스칼라 값은 소문자로 표기한다.
  - 따라서 벡터 X는 대문자를 사용하고 스칼라 y는 소문자를 사용하였다.
  - 사이킷런의 LinearRegression 객체는 가능한 직선들 가운데 데이터를 가장 잘 따르는 직선을 계산할 수 있다.
  - 이 선형회귀 함수의 기울기와 절편을 출력하고 관계 점수를 알아보자

```
coef = regr.coef_                  # 직선의 기울기
intercept = regr.intercept_        # 직선의 절편
score = regr.score(X, y)           # 학습된 직선이 데이터를 얼마나 잘 따르는가

print('y = {}* X + {:.2f}'.format(coef.round(2), intercept))
print('데이터와 선형 회귀 직선의 관계점수: {:.1%}'.format(score))
```

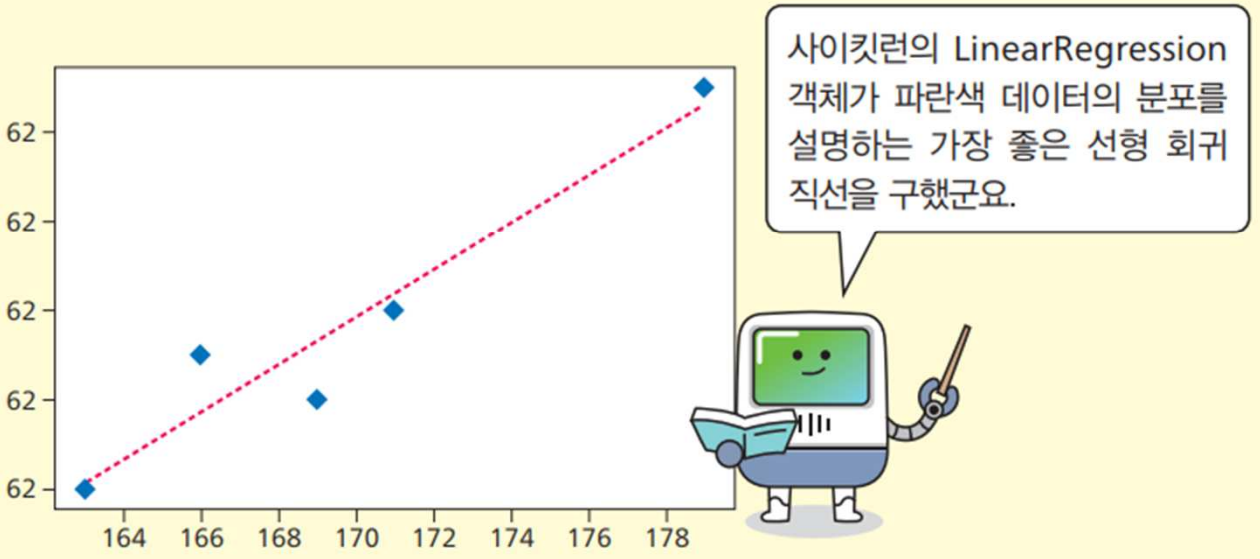
```
y = [0.53]* X + -32.50
데이터와 선형 회귀 직선의 관계점수: 91.9%
```

데이터를 시각화하고 차원을 증가시키자

- 시각화

```
import matplotlib.pyplot as plt

# 학습 데이터와 y값을 산포도로 그린다
plt.scatter(X, y, color='blue', marker='D')
# 학습 데이터를 입력으로 하여 예측값을 계산한다
y_pred = regr.predict(X)
# 계산된 기울기와 y 절편을 가지는 점선을 그려 보자
plt.plot(X, y_pred, 'r:')
```



데이터를 시각화하고 차원을 증가시키자

- 이제 이 모델에 다음과 같이 키가 167인 동민이의 키를 넣어서 그 추정값을 출력해 보자(출력 데이터의 차원은 일단 무시하자).
  - 이를 위하여 predict()라는 메소드를 사용하는 것을 볼 수 있다

```
unseen = [[167]]
result = regr.predict(unseen)
print('동민이의 키가 {}cm이므로 몸무게는 {}kg으로 추정됨'.format( \
      unseen, result.round(1)))
```

```
동민이의 키가 [[167]]cm이므로 몸무게는 [56.2]kg으로 추정됨
```



남자와 여자의 경우를 살펴보면 같은 키의 남자가 여자에 비하여 몸무게가 더 많이 나가는 경우가 많은데, 이 모델은 여자, 남자를 하나의 선형방정식으로 표현하므로 부정확하다는 생각이 드는 군요

데이터를 시각화하고 차원을 증가시키자

- 동민이네 반 학생들의 키와 몸무게를 다음과 같이 남학생 8명, 여학생 8명으로 나누어서 측정한 데이터를 새로 만들고 선형 회귀 모델에 적용해 보도록 하자.
  - 다음과 같은 데이터를 2 차원 입력 데이터 X로 만들자

남학생								
키	168	166	173	165	177	163	178	172
몸무게	65	61	68	63	68	61	76	67

여학생								
키	163	162	171	162	164	162	158	173
몸무게	55	51	59	53	61	56	44	57

```
X = [[168, 0], [166, 0], [173, 0], [165, 0], [177, 0], [163, 0], \
     [178, 0], [172, 0], [163, 1], [162, 1], [171, 1], [162, 1], \
     [164, 1], [162, 1], [158, 1], [173, 1]]      # 2차원 입력 데이터
```

데이터를 시각화하고 차원을 증가시키자

- 선형회귀 모델을 만들고 입력과 출력을 넣어주자.

```
regr = linear_model.LinearRegression()
X = [[168, 0], [166, 0], [173, 0], [165, 0], [177, 0], [163, 0], \
     [178, 0], [172, 0], [163, 1], [162, 1], [171, 1], [162, 1], \
     [164, 1], [162, 1], [158, 1], [173, 1]]      # 2차원 입력 데이터
y = [65, 61, 68, 63, 68, 61, 76, 67, 55, 51, 59, 53, 61, 56, 44, 57]
regr.fit(X, y)                                    # 학습시키기

print('계수 :', regr.coef_ )
print('절편 :', regr.intercept_)
print('점수 :', regr.score(X, y))
print('키 167cm 남학생 동민이의 추정 몸무게 :', regr.predict([[167, 0]]))
```

데이터를 시각화하고 차원을 증가시키자

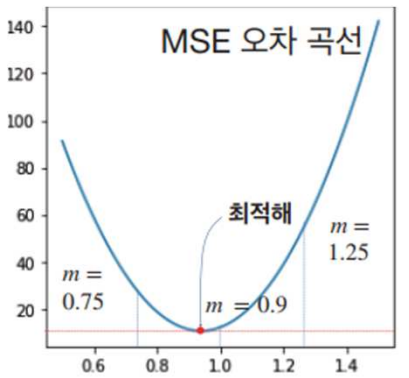
- 키가 167cm인 남학생/여학생의 추정 몸무게를 이 모델에게 물어보자

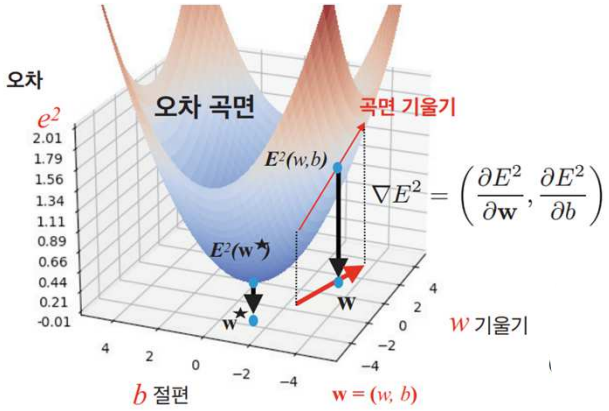
```
print('키 167cm 여학생 은지의 추정 몸무게 :', regr.predict([[167, 1]]))
```

```
계수 : [ 0.74803397 -7.23030041]
절편 : -61.22778389430634
점수 : 0.8425933302504423
키 167cm 남학생 동민이의 추정 몸무게 : [63.69388959]
키 167cm 여학생 은지의 추정 몸무게 : [56.46358918]
```

오차 곡면을 알아보자

- 경사하강법을 구현하기 위해 직선의 기울기에 대하여 오차를 제곱하여 오차 곡선의 기울기를 따라 내려가는 과정은 이미 살펴보았다.
  - 이 과정은 변수가 하나뿐이기 때문에 2차원 평면의 곡선으로 표현되었다.
  - 이제 직선의 기울기 ω와 절편 b 각각에 대해 오차의 제곱을 구해 보도록 하자.
  - 이전에는 직선의 기울기를 m으로 표기하였는데 앞으로는 가중치(weight)를 의미하는 ω로 표기할 것이다.
  - 곡선이 한 개가 아닌 두 개이므로 오차 곡면은 그림과 같이 밥그릇 모양을 하게 될 것이다





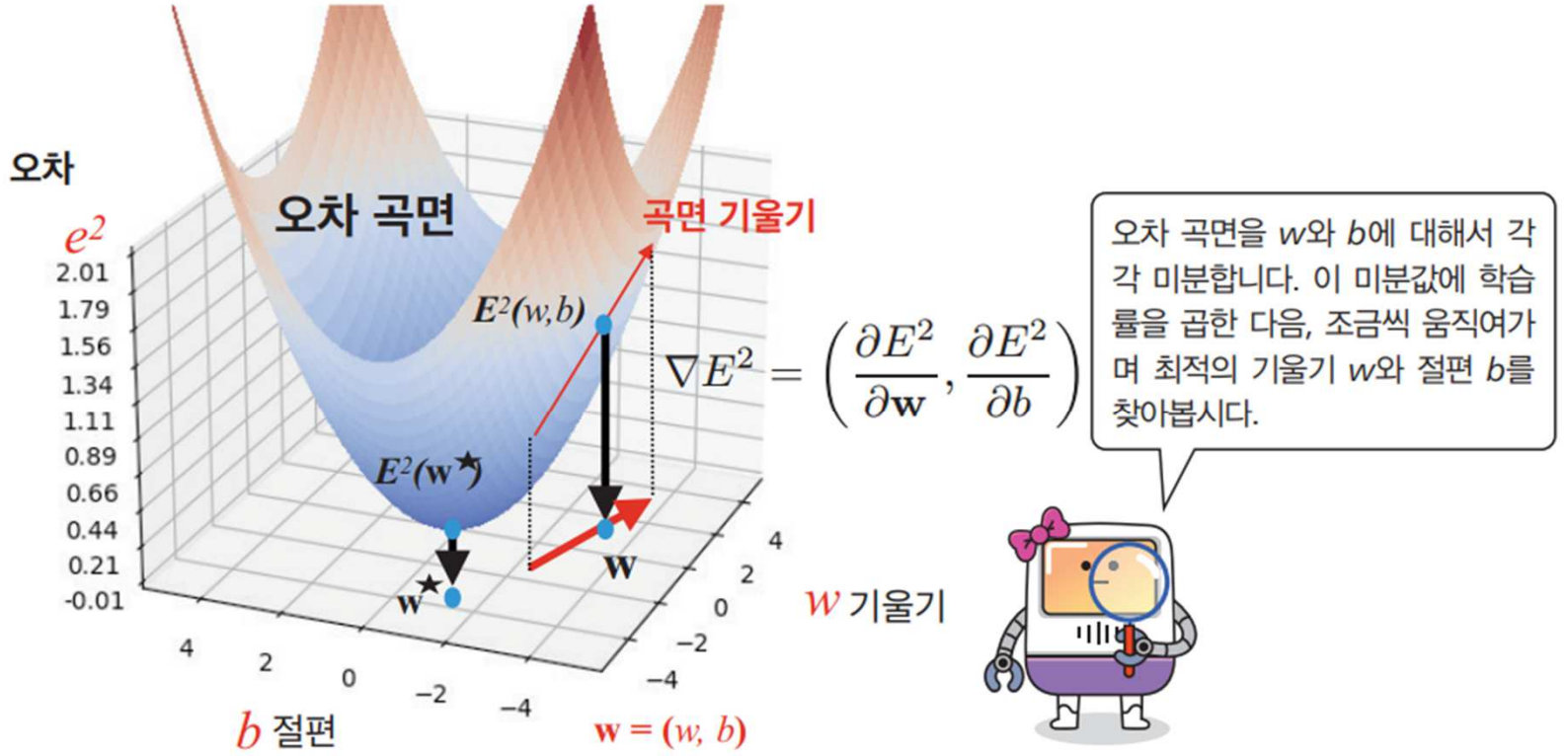
$$\nabla E^2 = \left(\frac{\partial E^2}{\partial \omega}, \frac{\partial E^2}{\partial b}\right)$$

$$\frac{\partial E^2}{\partial \omega} = \frac{\partial (\omega x + b - y)^2}{\partial \omega} = 2(\omega x + b - y)x = 2Ex$$

$$\frac{\partial E^2}{\partial b} = \frac{\partial (\omega x + b - y)^2}{\partial b} = 2(\omega x + b - y)\cdot 1 = 2E$$

오차 곡면을 알아보자

- 오차 곡면과 곡면의 기울기, 최적값



오차 곡면에서 최적의 기울기와 절편은 어디에 있나?

오차 곡면을 알아보자

- **하이퍼파라미터**
  - 머신러닝과 딥러닝에서 모델을 학습시키는데 필요한 파라미터
  - 하이퍼파라미터에는 학습률, 훈련 반복 횟수, 가중치 초기화 값들이 될 수 있다.
  - 여기서는 학습률 값을 0.005로 사용하였으며, 그리스 문자 η(에타)로 표기하였다
  - 오차를 E_i라고 할 때 다음과 같이 기울기 ω와 절편 b를 오차를 이용하여 수정할 수 있다
  - 이 때, 데이터별로 발생하는 오차를 하나씩 이용하여 갱신하는 것이 아니라, 벡터화 연산을 사용하여 아래와 같이 직선의 기울기와 절편을 갱신할 수 있다.
  - 이상의 내용을 조합하여 (1, 0), (4.5, 0.2), (9, 2.5), (10, 5.4), (13, 7.3)의 다섯 점들의 분포를 설명하는 선형 회귀 함수의 ω와 b를 찾아보도록 하자.
  - (2Ex, 2E)대신 (Ex, E)를 사용한 이유는 두 벡터의 크기는 다르지만 방향이 같기 때문이며, 어차피 이 벡터에 학습률을 곱할 것이기 때문에 상수 2가 불필요하기 때문이다.

$$\omega \leftarrow \omega - \eta \sum_{i=1}^{n} E_i x_i, \qquad b \leftarrow b - \eta \sum_{i=1}^{n} E_i$$

오차 곡면을 알아보자

- 에폭
  - 학습을 위해서는 전체 데이터를 모두 넣어서 에러를 구하는데, 이렇게 전체 데이터를 한 번 사용하는 것을 1 **에폭** epoch 이라 한다
  - w, b는 0으로 초기화 학습율learning_rate는 0.005, 에폭은 1000회로 두자
  - 다음과 같은 X, y가 있다. 이 데이터를 위한 선형회귀 함수의 w, b를 구하자

```
X = np.array([1, 4.5, 9, 10, 13])
y = np.array([0, 0.2, 2.5, 5.4, 7.3])

w, b = 0, 0                              # w, b의 초기값을 0으로 두자
learning_rate, epoch = 0.005, 1000       # 학습률과 학습 횟수(에폭)
n = len(X)                               # 입력 데이터 수
```

오차 곡면을 알아보자

- 에폭만큼 for 문을 수행하여 w, b를 구하자
  - y_pred는 현재의 w,b를 이용하여 추정한 값
  - y_pred-y는 실제 값과의 오차(error)
  - 이 오차를 이용하여 w, b를 개선함
  - 이 과정을 반복하자.

$$\omega \leftarrow \omega - \eta \sum_{i=1}^{n} E_i x_i, \qquad b \leftarrow b - \eta \sum_{i=1}^{n} E_i$$

```
for i in range(epoch):                  # 학습 루프
    y_pred = w*X + b                    # 현재 w, b를 이용한 작업 T
    error = y_pred - y                  # 성능 척도 P
    w = w - learning_rate * (error * X).sum()    # 경험 E로 개선
    b = b - learning_rate * error.sum()

print('w =', w.round(2), ', b =', b.round(2))
```

```
w = 0.63 , b = -1.65
```

경사하강법과 학습률

- 이전의 과정은 sklearn의 linear_model.LinearRegression으로 쉽게 얻을 수 있다.
  - 이전 결과와 동일한 w, b를 얻을 수 있다.

```
from sklearn import linear_model
import numpy as np

X = np.array([1, 4.5, 9, 10, 13])
y = np.array([0, 0.2, 2.5, 5.4, 7.3])
regr = linear_model.LinearRegression() # 절편값 b는 0으로 둔다
X = X[:, np.newaxis]
regr.fit(X, y)                         # 학습

print('w =', regr.coef_.round(2), \
      ', b =', regr.intercept_.round(2))
```

```
w = [0.63] , b = -1.65
```

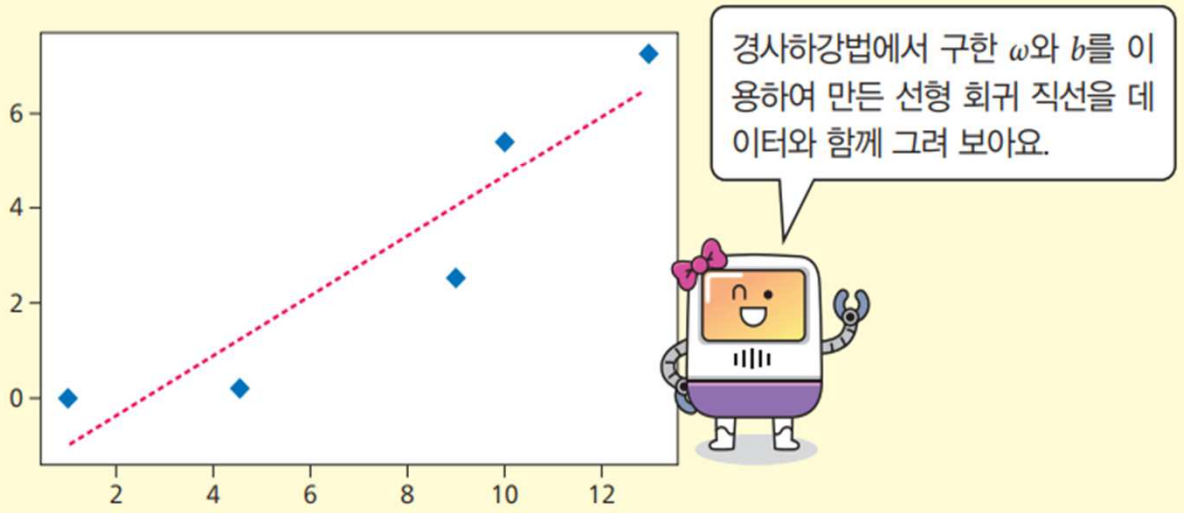
경사하강법과 학습률

- 시각화를 수행하자.
  - w=0.63
  - b=-1.65

```
import matplotlib.pyplot as plt
import numpy as np

X = np.array([1, 4.5, 9, 10, 13])
y = np.array([0, 0.2, 2.5, 5.4, 7.3])
plt.scatter(X, y, color='blue', marker='D')

# 계산으로 구한 w, b를 이용하여 선형 회귀 직선을 그리자
y_pred = 0.63 * X - 1.65
plt.plot(X, y_pred, 'r:')
```



경사하강법과 학습률

- 하이퍼파라미터 중 하나인 학습률을 매우 작은 0.00001로 둔다면 어떤 결과가 나타날까

```
# 학습률이 너무 작은 경우
X = np.array([1, 4.5, 9, 10, 13])
y = np.array([0, 0.2, 2.5, 5.4, 7.3])

w, b = 0, 0                                          # w, b의 초기값을 0으로 두자
learning_rate, epoch = 0.00001, 1000                 # 학습률과 학습 횟수(에폭)
n = len(X)                                           # 입력 데이터 수

for i in range(epoch):                               # 학습 루프
    y_pred = w*X + b                                 # 현재 w, b를 이용한 작업 T
    error = y_pred - y                               # 성능 척도 P
    w = w - learning_rate * (error * X).sum()        # 경험 E로 개선
    b = b - learning_rate * error.sum()

print('w =', w.round(2), ', b =', b.round(2))
```

```
w = 0.45 , b = 0.03
```

출력 결과와 같이 1,000회의 학습을 수행하였음에도 불구하고 학습한 ω와 b가 각각 0.45, 0.03으로 이전에 구한 0.63, −1.65와 차이가 발생하는 것을 볼 수 있다

경사하강법과 학습률

- 학습률을 1.0으로

```
# 학습률이 너무 큰 경우
X = np.array([1, 4.5, 9, 10, 13])
y = np.array([0, 0.2, 2.5, 5.4, 7.3])

w, b = 0, 0                                    # w, b의 초기값을 0으로 두자
learning_rate, epoch = 1.0, 1000               # 학습률과 학습 횟수(에폭)
n = len(X)                                     # 입력 데이터 수

for i in range(epoch):                         # 학습 루프
    y_pred = w*X + b                           # 현재 w, b를 이용한 작업 T
    error = y_pred - y                         # 성능 척도 P
    w = w - learning_rate * (error * X).sum()  # 경험 E로 개선
    b = b - learning_rate * error.sum()

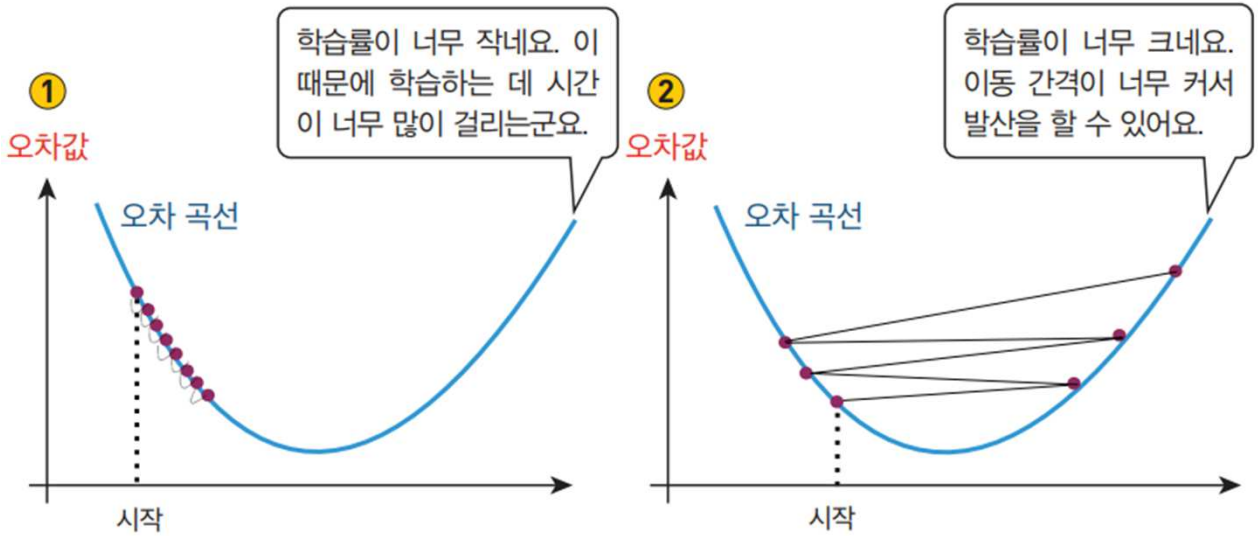
print('w =', w.round(2), ', b =', b.round(2))
```

```
w = nan , b = nan
<ipython-input-14-d045e867d86c>:12: RuntimeWarning: invalid value
 encountered in double_scalars
  w = w - learning_rate * (error * X).sum() # 경험 E로 개선
```

경사를 따라 이동하는 간격이 너무 커서 발산을 하게 되어 ω와 b를 제대로 얻지 못한다. 이로 인하여 ω와 b가 모두 Not a Number를 의미하는 nan 값을 출력한다.

경사하강법과 학습률

- 그림 1 은 오차 곡선을 타고 내려오는 경사하강법의 학습률이 너무 작은 경우이다.
  - 이 경우, 최적값에 도달하는데 너무나 많은 시간이 걸린다는 문제가 있다.
- 그림 2 는 그 반대의 경우로, 학습률이 너무 큰 경우이다.
  - 이동간격이 너무 커서 발산의 우려가 있다. 발산이 일어나게 되는 경우에도 학습이 제대로 이루어지지 않을 것이다.



학습률이 너무 작은 경우와 너무 큰 경우에 발생할 수 있는 문제점

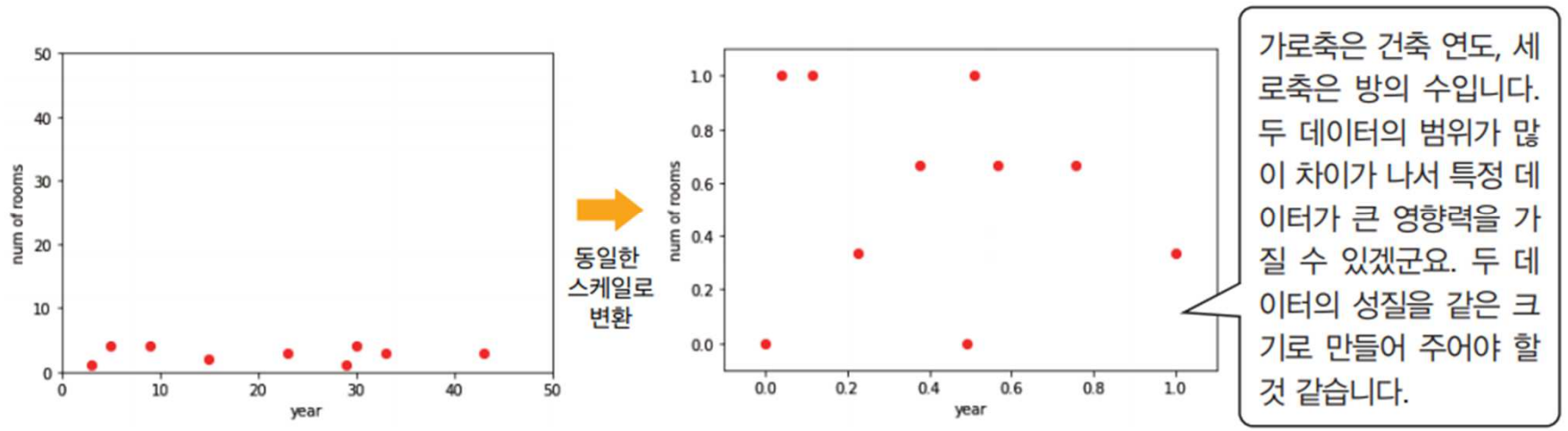
정규화와 표준화

- 정규화와 표준화의 필요성
  - 학습의 성능을 높이기 위해서는 초기화 방법과 활성화 함수, 개선된 경사하강법 그리고 보다 나은 오차계산 방법에 대한 이해가 필요하다.
  - 이러한 방법과 함께 입력 속성값의 범위를 비슷하게 만들어 주는 **정규화**와 **표준화** 도 필요하다
  - 머신러닝과 딥러닝에서 사용되는 데이터 값 중에서 균일하지 않은 **축적** scale 을 가지는 데이터를 우리 주변에서 많이 볼 수 있다.
    - 예를 들면, 대부분 1에서 80 사이에 분포하는 성질인 사람의 **나이** age 와 마이너스 수십억 원(부채가 있는 경우)에서 수백억 원까지의 값을 가지는 **개인 자산** personal assets 등이 그 좋은 예가 될 것이다.
  - 다음 표와 같이 집값에 영향을 미치는 속성값으로 **건축 연도**와 **방의 수**가 있다고 가정하자.
    - 우리는 다음과 같은 표를 통해 데이터를 표시할 수 있을 것이다. 편의상 가격의 단위는 억 원이라고 하자.

건축 연도	15	30	23	5	9	43	33	29	3	56
방의 수	2	4	3	4	4	3	3	1	1	2
가격	3.2	3.4	5	3.4	3.7	1.1	1.5	3.9	5.3	3.0

정규화와 표준화

- 주택의 건축 연도는 3년에서 56년으로 범위가 큰 반면, 방의 수는 한 개에서
- 네 개까지로 그다지 큰 범위를 나타내지 않는다. 위의 경우를 그래프로 그려보면 왼쪽과 같이 나타날 것이다.
  - 이 경우 딥러닝 알고리즘이 예측을 할 때 방의 개수보다는 더 큰 스케일을 가지는 건축 연도에 의하여 예측값이 좌지우지될 가능성이 크다. 따라서 이 두 개의 성질을 모두 오른쪽 그림과 같이 0에서 1 혹은 -1에서 1 사이의 값으로 변환시켜주는 것이 바람직할 것이다.



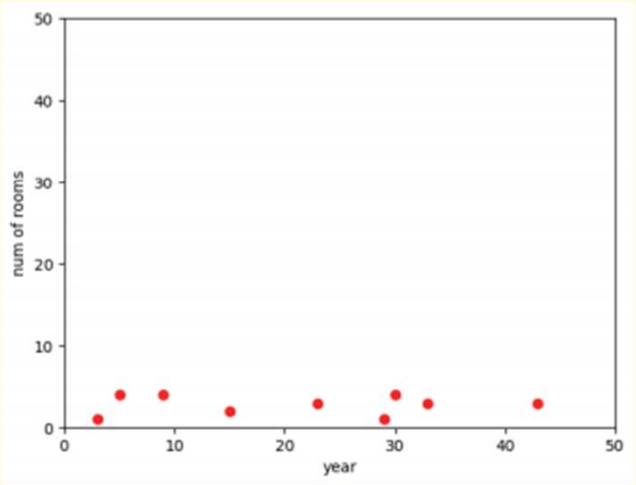
정규화와 표준화

- matplotlib을 사용
  - 시각화 시켜보기

```
import matplotlib.pyplot as plt

yr = [15, 30, 23, 5, 9, 43, 33, 29, 3, 56]
n_rooms = [2, 4, 3, 4, 4, 3, 3, 1, 1, 2]

plt.scatter(yr, n_rooms, c='red')
plt.xlim(0, 50)
plt.ylim(0, 50)
plt.xlabel('year')
plt.ylabel('num of rooms')
```



정규화와 표준화

- 그림을 살펴보면 대부분의 데이터가 바닥에 깔려 있는 형태로 존재한다.
  - 이와 같이 특성값의 범위가 다른 데이터는 경사하강법이나 딥러닝의 입력으로 바람직하지 못하다
  - 정규화의 필요성

- **특성 스케일링:** 데이터의 각 특성이 갖는 값의 범위가 다르면, 머신러닝 알고리즘은 스케일이 큰 특성에 더 큰 가중치를 두게 되는 경우가 많다. 이는 모델의 성능을 저하시킬 수 있다. 정규화를 통해 모든 특성의 스케일을 동일하게 만들어, 이러한 문제를 해결할 수 있다.
- **수렴 속도 개선:** 경사하강법 같은 최적화 알고리즘은 스케일이 비슷한 특성에서 더 빠르고 효과적으로 학습할 수 있다. 정규화를 통해 이러한 최적화 알고리즘의 수렴 속도를 개선할 수 있다.
- **수치 안정성 향상:** 정규화를 통해 모델의 수치 안정성이 향상되며, 이는 학습 과정에서 발생할 수 있는 수치 오류를 줄여준다.
- **모델 복잡도 감소:** 정규화는 모델이 복잡해지는 것을 방지하고, 오버피팅을 줄이는 데 도움이 된다. 이는 모델의 일반화 성능을 향상시키는 데 중요한 역할을 한다.

정규화와 표준화

- MinMaxScaler를 사용하자
  - 다음 식과 같이 입력을 조정
  - 최대값-최소값을 분모로
  - $x-x_{min}$을 분자로 두면 된다
  - 정규화 라고 한다

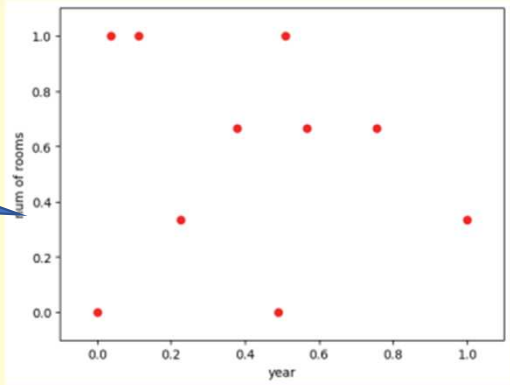
$$\tilde{x} = \frac{x - x_{\min}}{x_{\max} - x_{\min}}$$

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.preprocessing import MinMaxScaler

yr = np.array([15, 30, 23, 5, 9, 43, 33, 29, 3, 56])
n_rooms = np.array([2, 4, 3, 4, 4, 3, 3, 1, 1, 2])

min_max_scaler = MinMaxScaler()
yr_norm = min_max_scaler.fit_transform(yr[:, np.newaxis])
n_rooms_norm = min_max_scaler.fit_transform(n_rooms[:, np.newaxis])

plt.scatter(yr_norm, n_rooms_norm, c='red')
plt.xlim(-0.1, 1.1)
plt.ylim(-0.1, 1.1)
plt.xlabel('year')
plt.ylabel('num of rooms')
```



스케일링 된 데이터의 시각화 :
특성 값의 범위가 모두 0에서 1사이로 일정함

정규화와 표준화

- 정규화 이전과 이후의 값 살펴보기

```
import numpy as np

yr = np.array([15, 30, 23, 5, 9, 43, 33, 29, 3, 56])
n_rooms = np.array([2, 4, 3, 4, 4, 3, 3, 1, 1, 2])

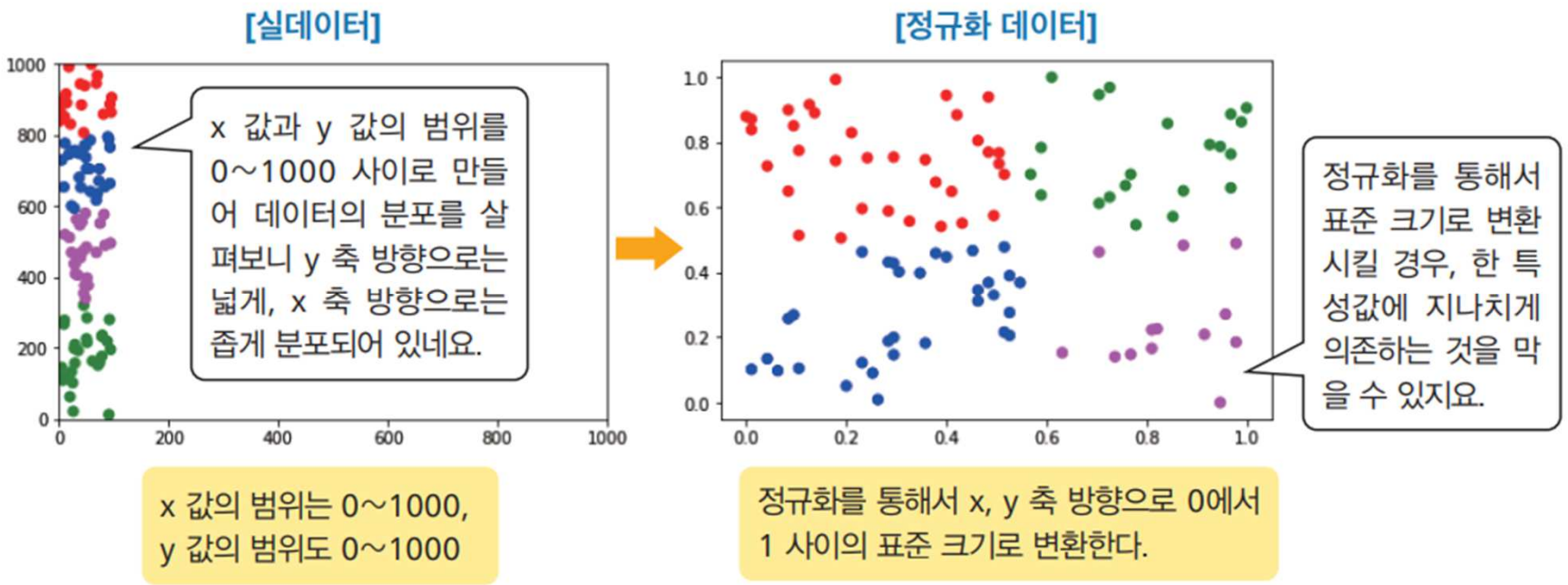
print(f'정규화 이전의 yr: {yr}')
sz = np.max(yr) - np.min(yr)
np.set_printoptions(precision=2) # 출력의 정밀도 설정
print(f'정규화 이후의 yr: {((yr - np.min(yr))/sz)}')

print(f'정규화 이전의 n_rooms: {n_rooms}')
sz = np.max(n_rooms) - np.min(n_rooms)
print(f'정규화 이후의 n_rooms: {((n_rooms - np.min(n_rooms))/sz)}')
```

```
정규화 이전의 yr: [15 30 23  5  9 43 33 29  3 56]
정규화 이후의 yr: [0.23 0.51 0.38 0.04 0.11 0.75 0.57 0.49 0.   1.  ]
정규화 이전의 n_rooms: [2 4 3 4 4 3 3 1 1 2]
정규화 이후의 n_rooms: [0.33 1.   0.67 1.   1.   0.67 0.67 0.   0.   0.33]
```

정규화와 표준화

- 정규화의 또 다른 사례 : 클러스터링



정규화와 표준화

- 표준화
  - 표준 스케일러를 사용
  - 평균이 0이고 표준편차가 1인 정규분포를 가지도록 데이터를 조정
  - 데이터의 평균이 μ_x
  - 표준편차가 σ_x일 때
  - 다음 식을 사용함

$$x' = \frac{x - \mu_x}{\sigma_x}$$

```
from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()
# 표준화 데이터를 생성함
yr_scaled_d = scaler.fit_transform(yr[:, np.newaxis])
print(f'표준화 이전의 yr: {yr}')
np.set_printoptions(precision=2)
print(f'표준화 이후의 yr: {yr_scaled_d}')
```

```
표준화 이전의 yr: [15 30 23  5  9 43 33 29  3 56]
표준화 이후의 yr: [[-0.59]
 [ 0.33]
 [-0.1 ]
 [-1.21]
 [-0.96]
 [ 1.13]
 [ 0.52]
 [ 0.27]
 [-1.33]
 [ 1.94]]
```

정규화와 표준화

- 표준화
  - 표준 스케일러를 사용
  - 평균이 0이고 표준편차가 1인 정규분포를 가지도록 데이터를 조정
  - 데이터의 평균이 μ_x
  - 표준편차가 σ_x일 때
  - 다음 식을 사용

$$x' = \frac{x - \mu_x}{\sigma_x}$$

```
from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()
# 표준화 데이터를 생성함
yr_scaled_d = scaler.fit_transform(yr[:, np.newaxis])
print(f'표준화 이전의 yr: {yr}')
np.set_printoptions(precision=2)
print(f'표준화 이후의 yr: {yr_scaled_d}')
```

```
표준화 이전의 yr: [15 30 23  5  9 43 33 29  3 56]
표준화 이후의 yr: [[-0.59]
 [ 0.33]
 [-0.1 ]
 [-1.21]
 [-0.96]
 [ 1.13]
 [ 0.52]
 [ 0.27]
 [-1.33]
 [ 1.94]]
```

```
print(f'표준화 이후의 yr: {yr_scaled_d.flatten()}')
```

```
표준화 이후의 yr: [-0.59 0.33 -0.1 -1.21 -0.96 1.13 0.52 0.27 -1.33 1.94]
```

요약

01 **회귀 분석**은 대표적인 지도학습 알고리즘으로, 관측된 데이터를 통해 독립변수와 종속변수 사이의 숨어 있는 관계를 추정하는 것이다.

02 **독립변수**는 가설의 원인이 되는 변수이며, **종속변수**에 영향을 미치는 선행조건을 뜻한다. **종속변수**는 가설의 결과가 되는 변수이며, 관측이나 측정이 가능한 변수이다.

03 **특성**이란 관찰되는 현상에서 측정할 수 있는 개별적인 속성을 의미한다. 머신러닝과 딥러닝에서는 학습의 결과를 결정하는 데에 영향을 미치는 입력 데이터를 말한다.

04 **선형 회귀**란 데이터를 설명하는 선형적인 관계를 모델링하는 것으로 변수 사이의 관계를 설명하는 **일차방정식에서 가장 적절한 기울기와 절편값을 찾는 것**이다.

05 **머신러닝**에서 말하는 좋은 모델이란 여러 모델들 중에서 이들의 예측값과 실제값의 차이인 오차가 가장 작은 모델이다.

06 오차의 합을 구하고 그것을 데이터의 개수로 나누어 평균을 구하여 얻은 오차값을 **평균 절대값 오차**라고 하고, 오차 제곱값의 평균으로 오차를 측정하여 얻은 오차값을 평균 제곱오차라고 한다. 회귀 모델의 오차는 일반적으로 **평균 제곱오차**를 널리 사용한다.

07 데이터의 분포를 가장 잘 설명하는 선형 회귀 직선을 구하는 과정을 **최적화**라고 한다. 가장 널리 알려진 최적화 알고리즘 중의 하나가 경사하강법이다.

08 머신러닝 라이브러리인 **사이킷런**은 지도학습, 비지도학습을 위한 다양한 모델을 제공하며, 이 모델을 위한 시각화 도구, 교차 검증 도구들까지 매우 광범위한 기능을 제공한다.

09 훈련 모델을 구현하기 위하여 모델에 설정되는 학습에 사용되는 파라미터를 **하이퍼파라미터**라고 한다. 하이퍼파라미터에는 **학습률**, **훈련 반복 횟수**, **가중치 초기화** 값들이 될 수 있다.

10 경사하강법에서 학습을 위하여 사용하는 학습률이 너무 작은 경우, 최적값에 도달하는데 너무나 많은 시간이 걸린다. 반대로 학습률이 너무 큰 경우, 이동 간격이 너무 커서 발산의 우려가 있다.